ABSTRACT

With the increasing focus on health and wellness, individuals often face difficulties in managing their weight, tracking their nutritional habits, and accessing expert medical advice—all in one place. The absence of an integrated digital solution leads to inconsistent health routines, lack of motivation, and limited professional support.

To address these challenges, this project introduces **Be Fit**, a comprehensive **fitness web application** designed to support users in achieving their health and wellness goals. **Be Fit** provides essential features such as **BMI analysis**, **personalized diet plans** for both weight loss and gain, and **consultation with certified physicians**, all integrated into a single platform.

Fit Hub distinguishes itself by going beyond basic fitness tracking. It offers **tailored health recommendations** based on user-specific factors such as BMI, age, gender, and activity level. Users can also benefit from **daily calorie tracking**, **nutrition logging**, and **progress monitoring**, ensuring consistency and accountability in their fitness journey.

To enhance the support system, Fit Hub includes a **real-time consultation module**, allowing users to schedule appointments with doctors for professional advice on managing conditions like diabetes, thyroid, or obesity-related issues. The platform also incorporates **secure user authentication**, a **role-based access system**, and a **user-friendly interface** that caters to different user types—Admin, Physician, and Patient.

By integrating health tools and expert guidance into a unified ecosystem, **Fit Hub** empowers users to take control of their fitness journey. Planned future enhancements include **AI-driven diet recommendations**, **interactive workout modules**, and **mobile app support**, making Fit Hub a dynamic and accessible solution for holistic health management.

TABLE OF CONTENT

1.INTODUCTION

In today's fast-paced world, maintaining a healthy lifestyle is more challenging than ever. Many individuals struggle with managing weight, eating nutritious meals, and accessing timely medical advice. These difficulties are often compounded by the absence of an integrated digital platform that offers a structured and personalized approach to health and fitness. This is where **Fit Hub** steps in.

Fit Hub is not just another fitness tracker—it is a smart, interactive, and comprehensive **fitness web application** designed to support individuals in their journey toward better health. Whether you are trying to lose weight, gain muscle, or simply lead a healthier life, Fit Hub provides a **personalized, user-friendly, and expert-backed environment** for users to track their fitness metrics, follow diet plans, and consult with certified doctors.

1.1 Overview of the Project

- **Fit Hub** is a web-based fitness application built using the MERN stack with Handlebars (HBS) for templating. It provides users with tools for BMI analysis, goal-based diet planning, and physician consultations, all through a single platform**.**
- **User-Friendly Interface** – Clean and responsive design for easy navigation across all devices.
- **BMI Calculation** – Allows users to input height and weight to receive instant BMI results and health status classification.
- **Goal-Based Diet Planning –** Users can choose between weight loss or gain, with personalized meal plans generated based on their age, gender, activity level, and preferences.
- **Calorie & Progress Tracking** – Enables users to log food intake, track calories, and monitor weight and BMI changes over time.
- **Physician Consultation Module** – Users can book appointments with doctors to receive expert advice on health-related concerns such as obesity, diabetes, or thyroid issues.
- **Notifications & Request Management** – Real-time alerts for collaboration requests, messages, and updates.
- **Secure Role-Based Access –** Implements Express-session for secure login and different roles for Admin, Physician, and User to manage their respective tasks efficiently.
- **Notifications & Appointment Management** – Real-time alerts for appointment bookings, physician responses, and user updates
- **Data Privacy & Security** – Ensures user data protection through session management and secure storage practices.

- **Future Expansions** – Plans include:
  - **AI-Based Diet & Workout Recommendations** – Personalized suggestions based on user progress and behavior.
  - **Mobile App Development** – Extending platform accessibility through Android and iOS apps.
  - **Interactive Workout Plans** – Video-based exercise guides and customized fitness schedules.

2. SYSTEM ANALYSIS

System analysis plays a crucial role in the **Software Development Life Cycle (SDLC)**, helping identify both the **functional** and **non-functional requirements** of the application. In the context of Fit Hub, this phase involves analyzing the challenges users face in maintaining their fitness routines, managing their diet, and accessing expert health guidance. The goal is to design a unified, secure, and interactive system that delivers a **personalized**, **doctor-assisted**, and **data-driven** health management experience.

2.1 Existing System

The current health and fitness ecosystem includes a range of apps for BMI calculators, diet tracking, or medical appointments—but most suffer from fragmentation and lack integration. Common issues include:

- **Scattered Tools:** Users must switch between different apps for BMI tracking, diet planning, and medical consultations, causing confusion and inefficiency.
- **Generic Diet Recommendations:** Most platforms provide one-size-fits-all diet charts, ignoring individual differences in age, health condition, and goals.
- **Limited Medical Support:** Few apps provide access to certified physicians, leaving users without proper medical advice, especially for conditions like thyroid, diabetes, or obesity.
- **No Real-Time Expert Guidance:** Platforms lack integrated doctor consultations and real-time appointment scheduling features.
- **Poor Data Security & Role Management:** Most systems fail to enforce secure authentication or differentiate between user roles like patients and doctors.

2.2 Proposed System

Fit Hub addresses these limitations through a comprehensive and integrated fitness management solution that combines personalized tools with expert health advice. The system includes:

- **BMI Analysis & Health Classification**:
    - Users can calculate their **Body Mass Index** by entering height and weight.
    - The system categorizes users as underweight, normal, overweight, or obese.
    - Results are used to drive personalized diet and consultation recommendations.

- **Personalized Diet Plan Generator**:
    - Based on the user's goal (weight gain or loss), the system creates tailored diet plans.
    - Considers **age, gender, activity level**, and **diet preferences** for accuracy.
    - Includes **daily calorie tracking** and food log features to ensure accountability.
- **Doctor Consultation Module**:
    - Users can **book appointments** with certified physicians.
    - Doctors can **review BMI data**, health goals, and provide lifestyle guidance.
    - Encourages medically supported fitness journeys for users with special conditions.
- **Authentication & Role-Based Access**:
    - Secure **Express-session-based login system**.
    - Role-based access differentiates.
        - **Admin** – Manages the system and physicians
        - **Physician** – Reviews users' health status and offers medical advice.
        - **User** – Tracks BMI, diet, books doctor consultations.
    - Ensures privacy, user accountability, and streamlined navigation.
- **Interactive & User-Centric Interface**:
    - Clean, responsive UI optimized for mobile and desktop users.
    - Real-time alerts for **appointment confirmations**, **progress updates**, and **diet reminders**.
- **Scalability & Future Enhancements**:

Fit Hub is built with future-ready architecture for easy module expansion:

- AI-based diet and workout recommendations.
- Interactive workout video modules.
- **Gamification features** (e.g., points, badges, leaderboards).
- **Mobile app development** for iOS and Android.
- **Community support forums** or peer tracking tools

OPERATIONAL FEASIBILITY

Operational feasibility evaluates how practical, effective, and valuable the proposed system will be for its end users—in this case, individuals seeking to manage their health, improve fitness, and access expert medical advice. The **Fit Hub** platform demonstrates strong operational feasibility due to the following key factors:

- **Enhanced User Convenience:**
  - Users can access **BMI tracking**, **personalized diet plans**, and **physician consultations** all within a single platform, eliminating the need for multiple apps or services.
  - The **appointment booking system** and **real-time health tracking** features ensure a seamless and efficient experience for users pursuing their wellness goals.
- **Flexibility & Personalization:**
  - Users can **customize their fitness goals** (weight loss or gain), **track daily calorie intake**, and **monitor BMI changes**.
  - The platform offers **personalized diet plans** based on individual metrics like age, gender, activity level, and dietary preferences.
  - Physicians can **tailor medical recommendations** based on the user's health status and history.
- **Improved Health Support Experience:**
  - **Smart classification** of BMI ranges helps users understand their current condition and what actions to take.
  - Access to certified physicians adds credibility and ensures that users receive trusted, professional health advice.
  - **Real-time progress tracking** keeps users motivated and aligned with their goals
- **Seamless Integration with Healthcare & Wellness Ecosystem:**
  - The system can be extended to **integrate with fitness wearables**, nutrition APIs, or health records in the future.
  - Doctors and users can manage **appointments**, **updates**, and **medical advice** efficiently through a single portal.
  - Future plans include **video consultations**, **interactive workout modules**, and **dietitian dashboards**.
- **Minimal Learning Curve:**
  - The clean, responsive user interface is designed for users of all ages and technical backgrounds.

- Users can easily navigate through BMI calculators, diet modules, and doctor consultations without technical complexity.
- Built-in **help guides**, **FAQs**, and **user support channels** ensure smooth onboarding and ongoing assistance.

CONTENT SOURCES

To deliver accurate health recommendations, secure user interactions, and seamless platform functionality, the **Fit Hub** system relies on a variety of data and content sources, including:

- **Internal Databases**
  - Securely stores **user profiles**, **BMI records**, **diet plans**, **appointment bookings**, and **progress logs**
  - Maintains separate access layers for **Admins**, **Physicians**, and **Users** based on their roles
- **Real-Time Data**
  - **User activity status** (online/offline).
  - Appointment confirmations and reminders
  - Diet log submissions
  - BMI updates and progress records
- **User Input & Logs**
  - Login and authentication attempts
  - Profile updates
  - Diet preference changes
  - Appointment bookings and cancellations
  - Physician responses and recommendations
- **Third-Party Integrations**.
  - Email verification and notifications
  - Secure session handling
  - **Planned integration** with wearable fitness devices and third-party nutrition databases

3. PROBLEM DESCRIPTION

The current health and fitness application landscape lacks a **comprehensive**, **personalized**, and **medically-supported** platform that empowers users to manage their weight, diet, and health conditions effectively. Many platforms provide isolated solutions such as BMI calculators or meal planners, but fail to integrate them into a unified and user-friendly system.

This section outlines the key challenges faced by users and how **Fit Hub** is designed to address them

.

3.1 PROBLEM STATEMENT

The absence of an all-in-one, medically-integrated fitness platform introduces several limitations for health-conscious users:

- **Fragmented Health Tools**
  Users have to rely on separate apps for BMI tracking, diet planning, and medical consultations, which leads to inconsistency and poor health outcomes.
- **Lack of Personalized Recommendations**
  Most platforms offer generic health tips or diet plans that ignore individual factors such as age, BMI, lifestyle, or medical conditions.
- **Inadequate Medical Support**
  Current systems lack the integration of certified physicians, making it difficult for users to get expert advice tailored to their health profile.
- **Limited Progress Tracking**
  Users often struggle to monitor their long-term progress or view trends in their BMI, weight, or calorie intake due to weak tracking tools.
- **Poor User Experience**
  Many apps lack intuitive design, structured features, or clear workflows, reducing user engagement and retention.
- **Data Security & Role Mismanagement**
  Weak authentication and lack of role-based control raise concerns about the privacy of sensitive health data and medical consultations.

3.2 PROBLEM SOLUTION

Fit Hub is purpose-built to address the above challenges through a **feature-rich, secure, and user-focused** approach:

- **Integrated Health Management**
  Combines BMI tracking, personalized diet planning, and certified physician consultations into one unified platform**.**
- **Tailored Health Guidance**
  Generates diet plans and fitness suggestions based on personal metrics like age, gender, activity level, and BMI classification.
- **Doctor Consultation System**
  Allows users to book real-time appointments with doctors for expert advice on conditions such as thyroid, diabetes, or obesity.
- **Dynamic Progress Tracker**
  Enables users to log and visualize BMI, weight, calorie intake, and dietary changes over time for motivation and improvement.
- **Modern & Accessible UI**
  Responsive, intuitive interface ensures users of all skill levels can navigate and use the platform easily across devices.
- **Scalable Architecture**
  Built on the MERN stack with Handlebars, Fit Hub uses a modular, optimized backend and database system that supports high traffic with low latency.
- **Secure Authentication & Role Management**
  Uses Express-session authentication and defines clear roles (Admin, Physician, User) to ensure privacy, accountability, and controlled access.

4. SYSTEM SPECIFICATION

4.1 Hardware Requirements

The system requires the following hardware components for optimal performance:

- **Processor:** Intel Core i5 or higher (or equivalent AMD processor)
- **RAM:** Minimum 8GB (16GB recommended for better performance)
- **Storage:** Minimum 256GB SSD (Recommended: 512GB SSD or higher for faster processing)
- **Graphics:** Integrated or dedicated GPU for smooth UI rendering
- **Internet Connection:** Stable broadband connection for seamless database interactions
- **Server Requirements:** Cloud-based or dedicated server with scalable configurations

4.2 Software Requirements

The system is developed using the **MERN (MongoDB, Express.js, React, Node.js) stack**, requiring the following software components:

- **Operating System:** Windows, macOS, or Linux
- **Frontend:** React.js for a dynamic and responsive user interface
- **Backend:** Node.js with Express.js for efficient server-side operations
- **Database:** MongoDB for flexible and scalable data storage
- **Package Manager:** npm (Node Package Manager) for dependency management
- **Version Control:**
    - **Git:** For tracking changes in the source code
    - **GitHub:** For code collaboration, repository management, and deployment automation
- **Development Tools:**
    - VS Code (or any preferred IDE)
    - Postman for API testing
    - MongoDB Compass for database management
- **Security Implementations:**
    - JWT authentication for secure user access
    - Bcrypt for password hashing
    - HTTPS encryption for secure data transmission

This system specification ensures **smooth performance, scalability, and security** for the "Easy Search" project.

5. SYSTEM DESIGN

5.1 Infrastructure Design

The infrastructure design of **Fit Hub** focuses on delivering a **scalable**, **secure**, and **responsive** health and fitness management system. It is structured into three major components: **Frontend (Client-Side)**, **Backend (Server-Side)**, and **Database Design**. Together, these components ensure smooth functionality for BMI analysis, diet tracking, appointment scheduling, role-based access, and user progress monitoring.

1. **Frontend Architecture (Client-Side)**

   The frontend of Fit Hub is developed using **React.js**, a modern JavaScript library known for its performance and reusability. It is responsible for user interaction, rendering UI components, and communicating with the backend.

   - **User-Friendly Interface**:
     Clean, responsive, and mobile-friendly UI designed for users of all ages and backgrounds.
   - **Component-Based Design**:
     Modular React components for reusable and easily maintainable UI elements such as forms, dashboards, and appointment pages.
   - State Management:
     - Utilizes **Context API** or **Redux** for managing global states like user sessions, BMI data, diet preferences, and doctor availability.
     - Facilitates seamless data flow across the application.
   - API Communication:
     - Uses **Axios** or **Fetch API** to interact with backend services.
     - Handles tasks like user login, BMI updates, diet fetch requests, and appointment bookings.
   - **Authentication & Access Control**:
     - JWT-based login system to authenticate users securely.
     - Role-based rendering for Admin, Physician, and User dashboards.

2. **Backend Architecture (Server-Side)**

   The backend is built using **Node.js** with **Express.js**, providing a fast and flexible server-side environment. It manages all core application logic, data processing, and route handling.

   - **RESTful APIs**:

Provides endpoints for BMI calculation, diet plan generation, user registration, login, appointment management, and more.

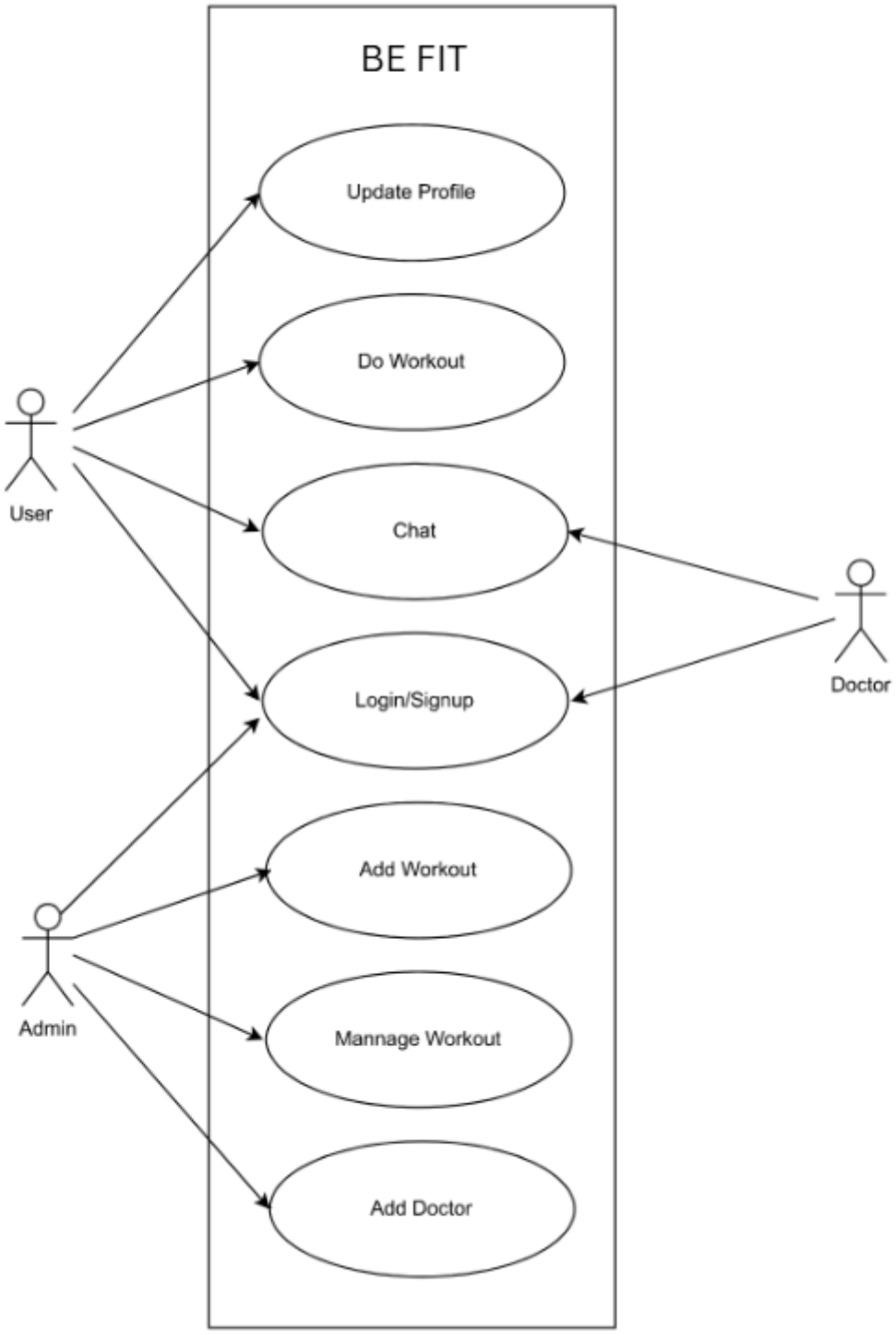
- **Authentication & Security**:
  - Uses **Express-session** or **JWT** for secure login and session management
  - Implements **bcrypt.js** for password hashing.
  - Enforces **CORS policies** to protect against cross-origin attacks.
- **Error Handling & Logging**:
  - Middleware handles invalid input, API errors, and failed requests.
  - Uses tools like **Winston** or **Morgan** for logging and diagnostics.

3. Database Design

Fit Hub uses **MongoDB**, a flexible and scalable NoSQL database ideal for storing structured and unstructured fitness and medical data.

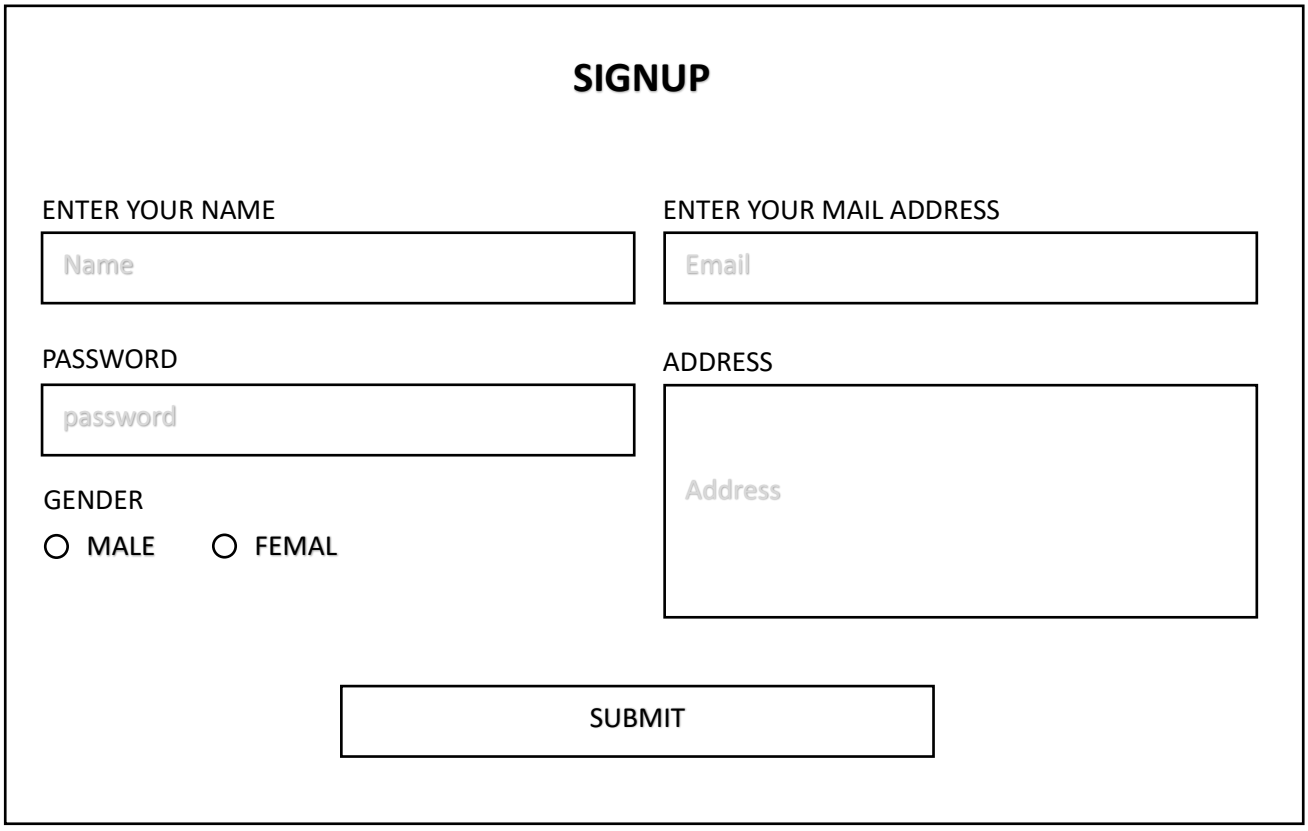
- **Users Collection**:
  Stores credentials, role, BMI history, preferences, and session data.
- **Diet Plans Collection**:
  Contains pre-configured and dynamically generated diet charts for weight loss and gain.
- Appointments Collection:
  Tracks doctor availability, appointment status, and consultation history
- Progress Logs:
  Records users' weight, BMI changes, and calorie intake for visualization and tracking
- **Optimization & Performance**:
  - Indexing for faster search/filtering (e.g., by date, user ID, or doctor ID)
  - Deployed using **MongoDB Atlas** for cloud scalability, automated backups, and availability

5.2 USE CASE DAIGRAM

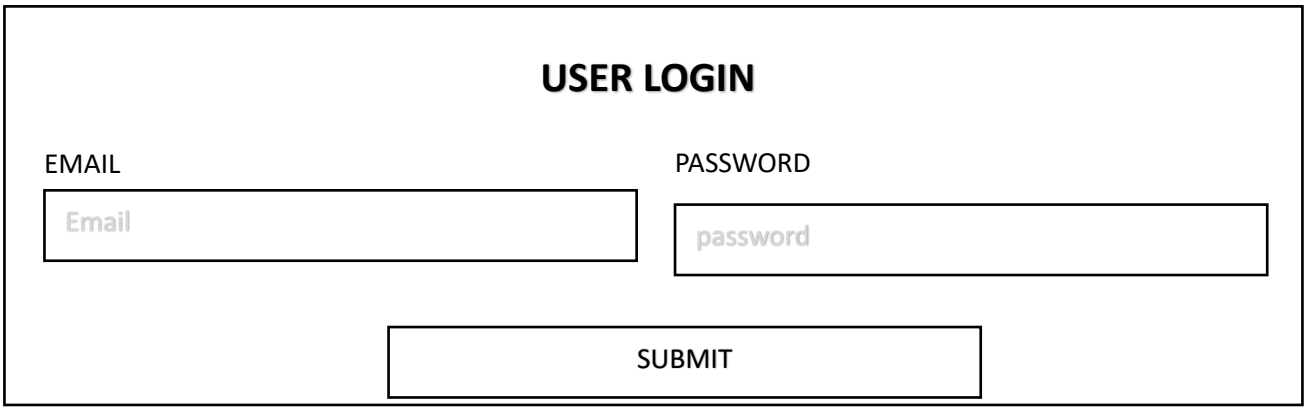


5.3 FORM DESIGN

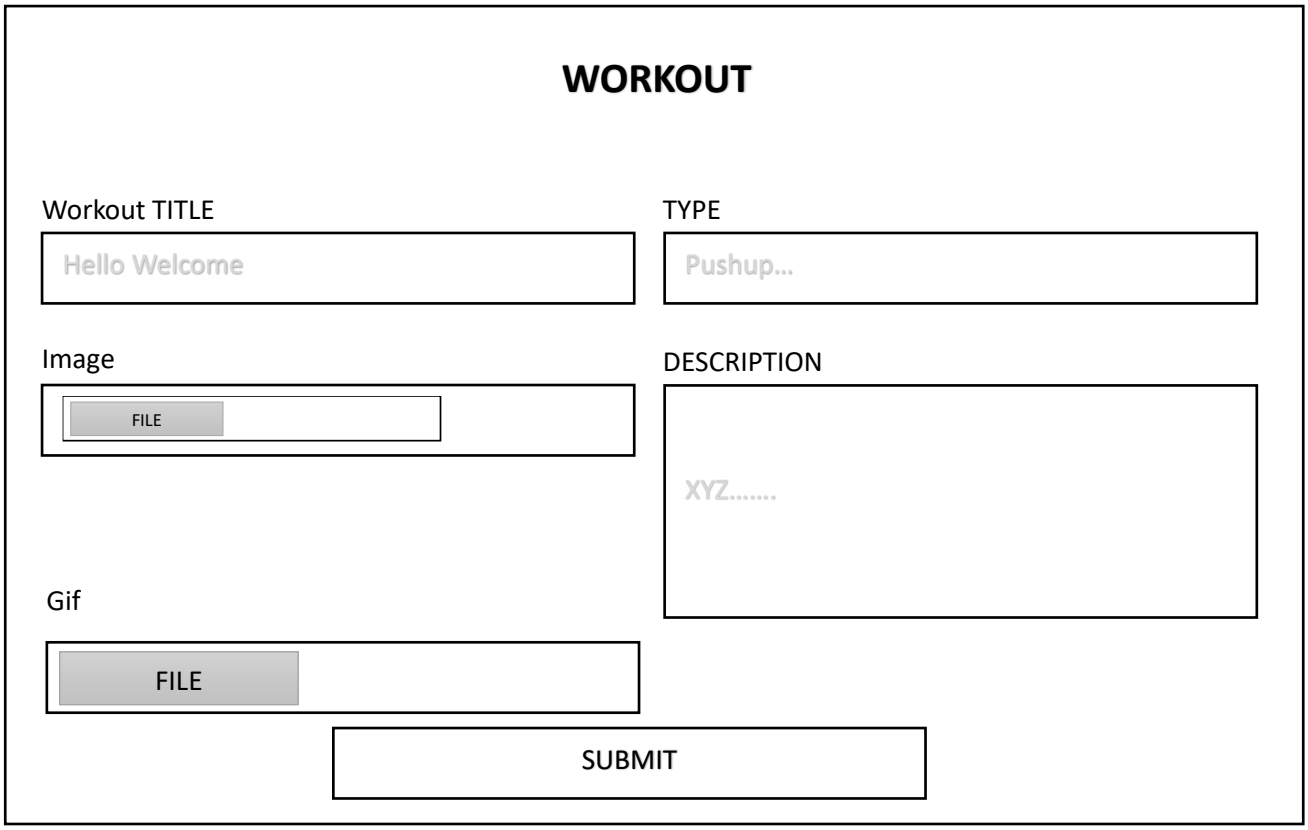
1.User Registration Form



2.User Login



3.Add Workout Form



5.4 JSON DESIGN

MongoDB JSON Design Description

MongoDB is a NoSQL, document-oriented database that stores data in a Binary JSON (BSON) format. Unlike relational databases that use tables and rows, MongoDB stores data in collections consisting of flexible JSON-like documents.

This design is highly scalable, schema-less, and optimized for high-performance applications, making it an ideal choice for MERN (MongoDB, Express, React, Node.js) stack development**.**

1.Admin JSON

- **Purpose**: Used to manage and represent information related to system administrators or users**.**
- **Structure:**

```
{
"_id":  "ObjectId",
"fullName": "String",
```

"username": "String",

"password": "String",

"__v": "Integer"

}

2. User JSON

- **Purpose:** Represents information related to users who interact with the system.
- **Structure:**

{

"_id": "ObjectId",

"fullName": "String",

"username": "String",

"email": "String",

"password": "String",

"phoneNumber": "String",

"createdAt": "Date",

"updatedAt": "Date",

"__v": "Integer"

}

3. Workout JSON

- **Purpose**: Represents information related to shops that are part of the system.
- **Structure:**

{

"_id": "ObjectId",

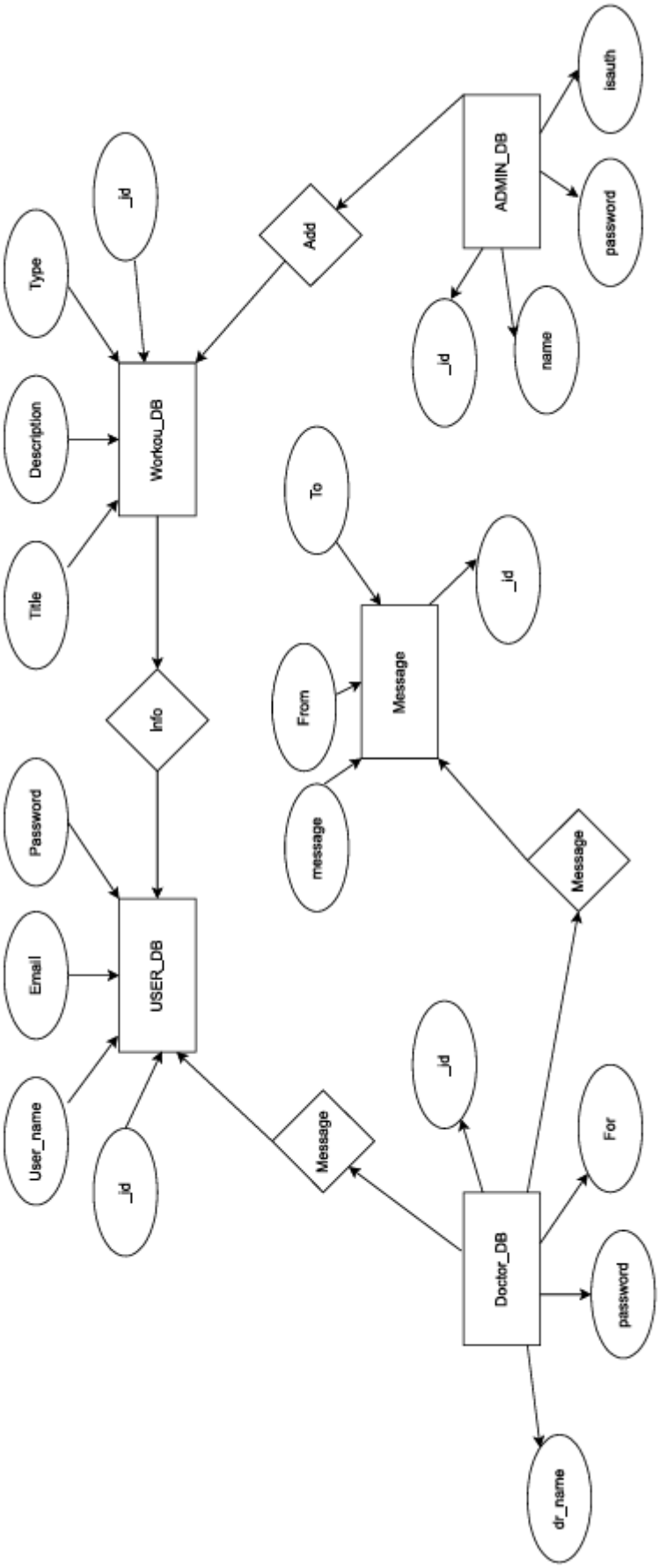
"workoutname": "String",

"type": "String",

"description": "String",

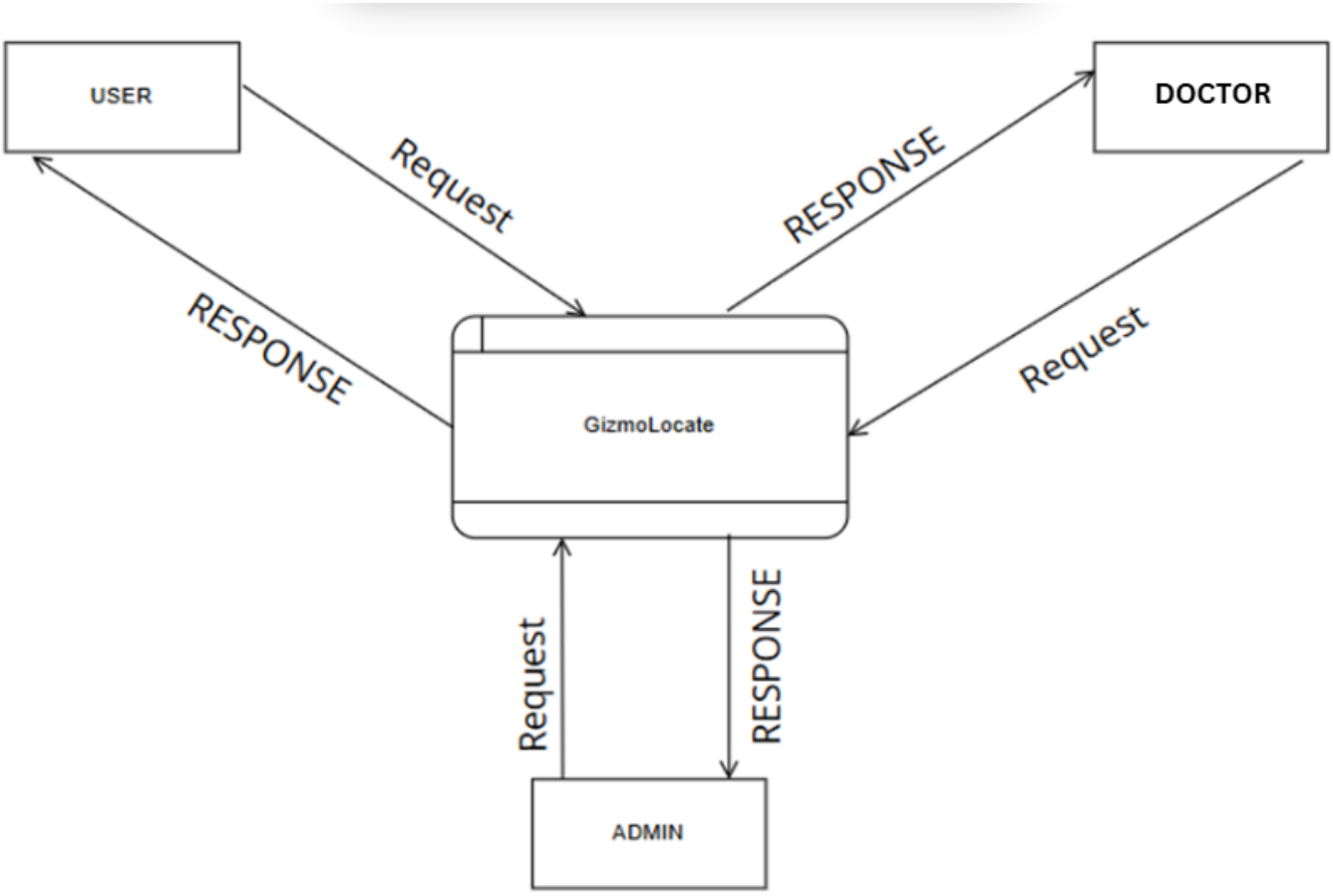
```
  "rep": "Number",
  "__v": "Integer"
}
```

5.6 ER DAIGRAM

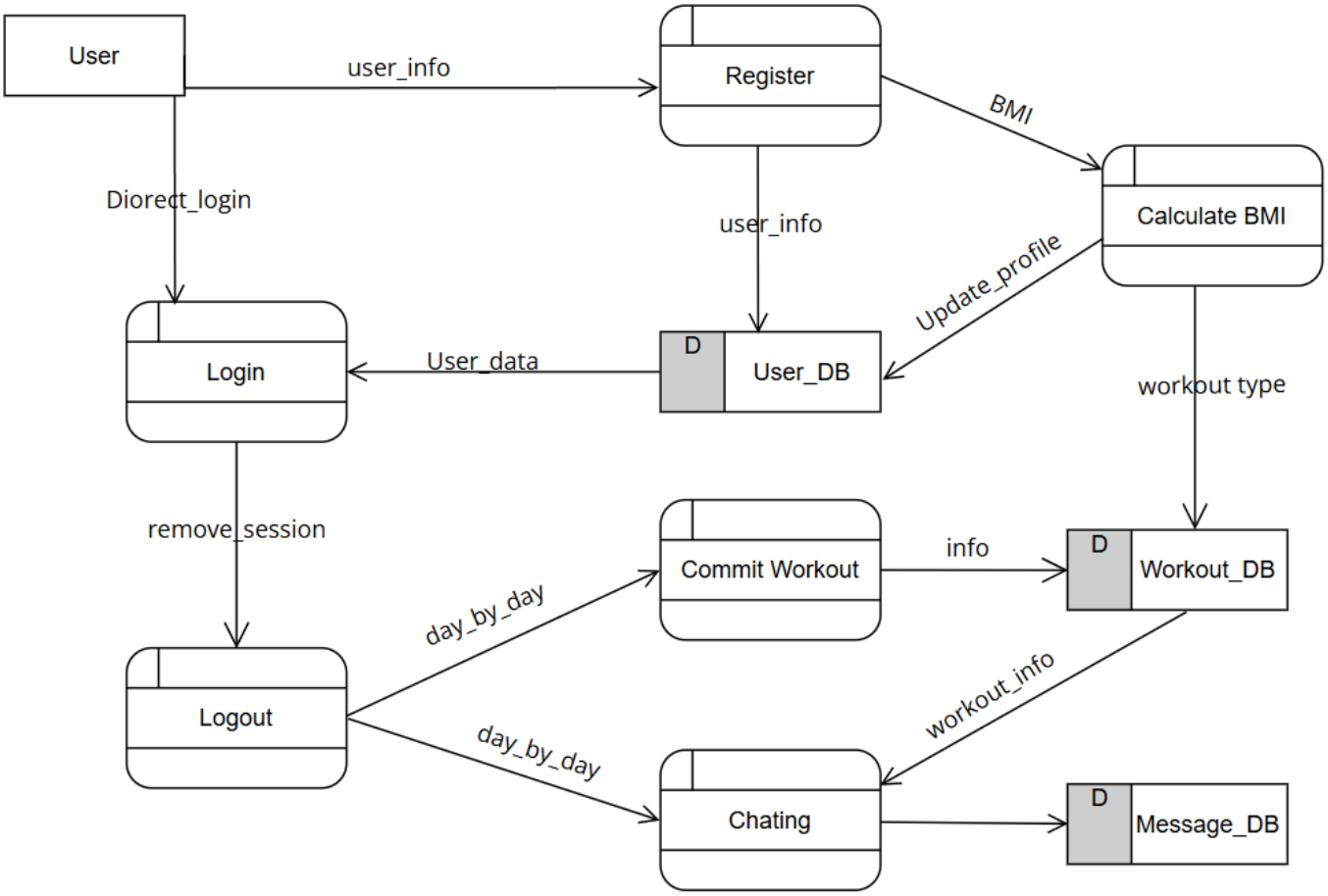


5.7 DATA FLOW DAIGRAM

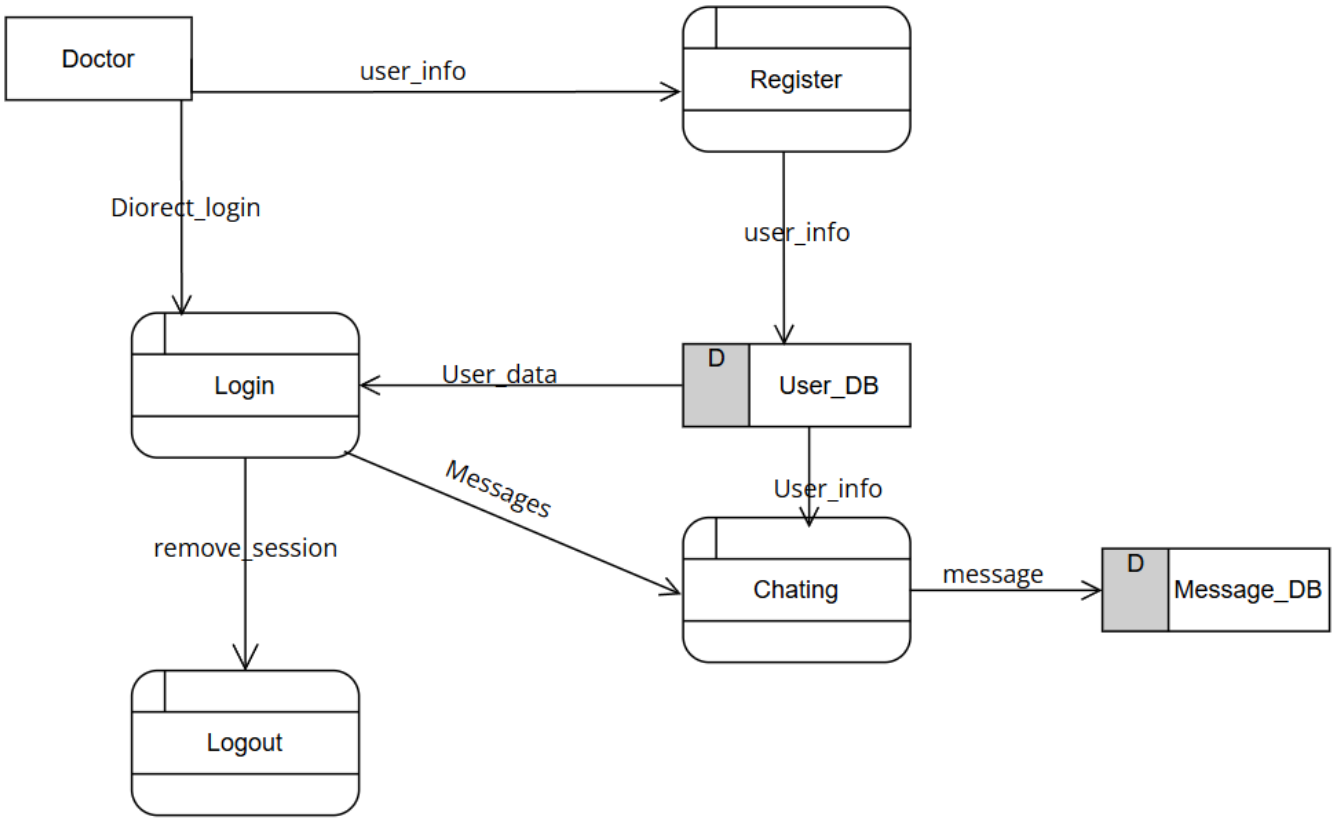
LEVEL 0



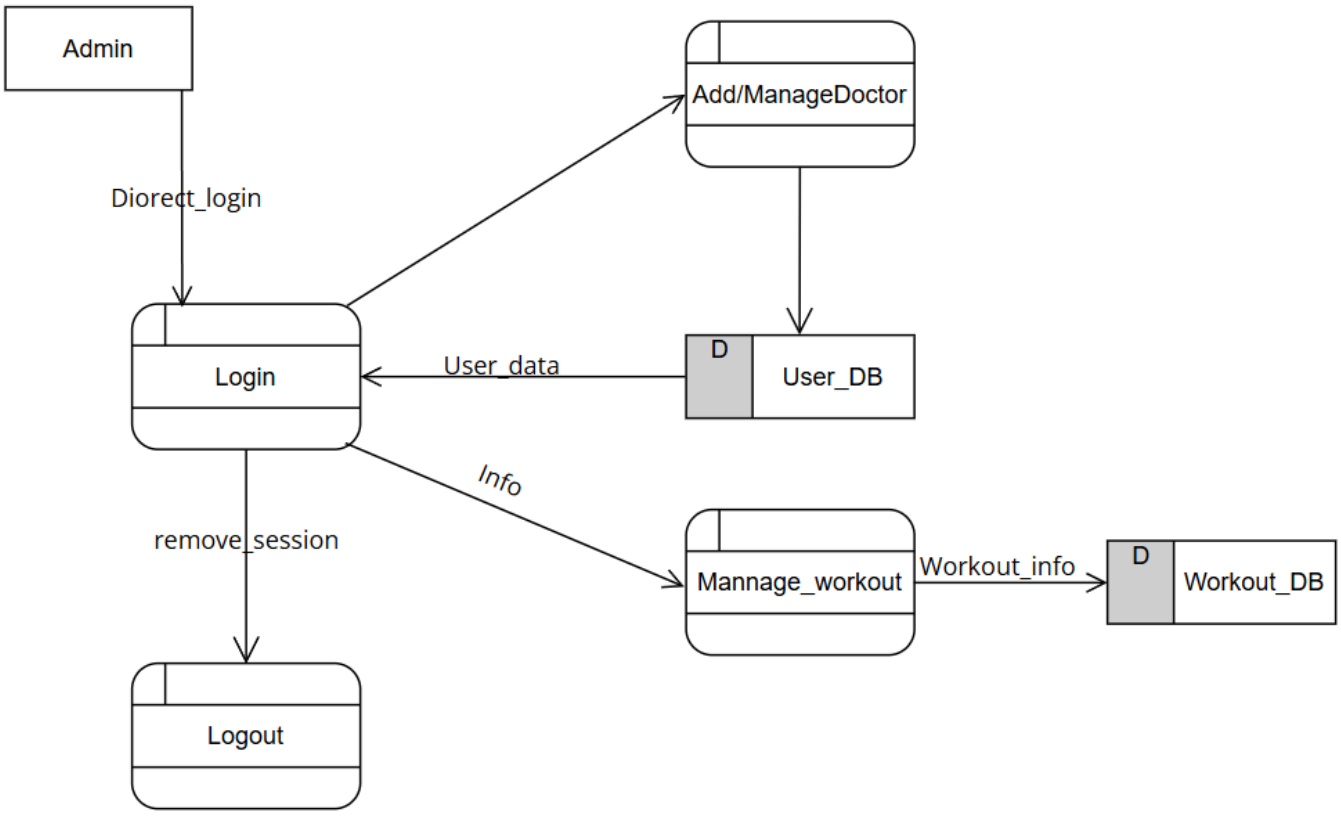
LEVEL 1: USER


User
user_info
Register
BMI
Calculate BMI
Diorect_login
user_info
Update_profile
Login
User_data
D
User_DB
workout type
remove_session
Commit Workout
info
D
Workout_DB
day_by_day
Logout
day_by_day
workout_info
Chating
D
Message_DB

LEVEL 1: DOCTOR



LEVEL 1: ADMIN



6. System Testing

System testing is a crucial phase in the **Fit Hub** development process to ensure that all modules—from BMI calculation to doctor consultations—function as intended and meet the specified user requirements. It involves several testing methodologies, including **Validation Testing**, **Black Box Testing**, and **White Box Testing**, to confirm both functional and non-functional correctness.

6.1 Testing methods

Validation Testing

- Confirms that the entire Fit Hub system satisfies all functional and non-functional requirements.
- Validates end-to-end workflows including user registration, diet recommendations, progress tracking, and appointment scheduling.
- Ensures the platform behaves according to user expectations and project objective.

Black Box Testing

- Conducted without knowledge of internal code or logic.
- Focuses on verifying.
    - Input/output correctness (e.g., BMI result accuracy)
    - User interface responses
    - Error handling for invalid inputs (e.g., wrong login)
- Used to test functionality such as user login, diet plan generation, and doctor appointment booking.

White Box Testing

- Focuses on the internal logic and code structure.
- Includes:
    - **Unit Testing** for functions like BMI formula, diet plan engine
    - **Integration Testing** across modules (e.g., BMI → Diet → Appointment)
    - **Code Coverage Analysis** to ensure maximum code path execution
- Helps detect logic flaws, security vulnerabilities, and performance bottlenecks.

6.2 Test Case

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC_01	User Login Validation	1. Navigate to login page2. Enter valid credentials3. Click "Login"	User logs in and is redirected to their dashboard	Pass
TC_02	Invalid User Login	1. Navigate to login page2. Enter wrong credentials3. Click "Login"	Error message: "Invalid Username or Password"	Pass
TC_03	BMI Calculation	1. Enter valid height and weight2. Click "Calculate BMI"	BMI result and health category displayed (e.g., Overweight)	Pass
TC_04	Diet Plan Generation	1. Select weight goal (Loss/Gain)2. Submit profile details	Personalized diet plan is generated	Pass
TC_05	Book Doctor Appointment	1. Navigate to Doctor List2. Select doctor3. Choose time slot4. Click "Book"	Appointment is booked and appears in user’s appointment list	Pass
TC_06	Real-Time Appointment Notification	1. Book an appointment2. Login as doctor	Doctor receives real-time notification of new booking	Pass
TC_07	System Security (Injection Attack Test)	1. Enter a malicious script in form fields2. Submit	System blocks malicious input and displays a generic error	Pass
TC_08	Profile Update	1. Navigate to "Edit Profile"2. Change data3. Click "Save"	Updated data reflects in user profile	Pass
TC_09	Role-Based Access (Admin)	1. Login as admin2. Navigate to admin tools	Admin dashboard loads; unauthorized users denied access	Pass

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC_10	Appointment Status Update	1. Doctor marks an appointment as complete	Status updated in both doctor and patient dashboards	Pass

7. System Implementation

System implementation is the process of integrating and deploying the developed system into a real-world working environment. It ensures that all components function correctly and efficiently according to the project's specifications. The implementation phase involves installing, configuring, and making the system available for end-users.

7.1 Deployment Strategy

The deployment of the system follows a structured approach to ensure minimal downtime and a smooth transition. Some common deployment strategies include:

- **Direct Deployment**: The new system fully replaces the old system immediately.
- **Parallel Deployment**: The new system runs alongside the old one for a period to ensure stability.
- **Phased Deployment**: The system is introduced in stages, module by module.
- **Pilot Deployment**: The system is deployed to a limited group of users before full implementation.

For this project, the **Phased Deployment** approach is recommended to minimize risk and allow for gradual adaptation.

7.2 Installation and Configuration

The system is built using the **MERN stack (MongoDB, Express.js, React, and Node.js)** and requires proper installation of its components:

1. **Backend Setup (Node.js & Express.js)**
    - Install **Node.js** and set up the server.
    - Install dependencies such as express, mongoose, and cors.
    - Connect the backend to the MongoDB database.
    - Set up authentication using JWT (JSON Web Tokens).
2. **Frontend Setup (React.js)**
    - Install **React.js** and set up components for UI.
    - Implement state management using **Redux** or **Context API**.
    - Configure API calls to interact with the backend.

3. **Database Setup (MongoDB)**
    - Create collections for **users, doctor, admin, and workouts**.
    - Implement indexing for faster search and retrieval.
    - Ensure **data validation** and security measures.
    - Use **MongoDB Atlas** for cloud-based database management.

7.3 Data Migration

If migrating from an existing system, data migration is performed to transfer old records to the new system. Steps include:

1. **Extracting Data** from the old system.
2. **Transforming Data** to match the new system's structure.
3. **Loading Data** into the MongoDB database.
4. **Testing Data Integrity** to ensure no data loss.

7.4 User Training and Support

To ensure users can efficiently operate the system, training sessions are conducted, including:

- **Admin Training**: Managing workout, Add Workout.
- **doctor Training**: Chatting with users.
- **User Training**: Navigating the system, Do Workouts.

A support team is available for troubleshooting issues, and documentation is provided for self-learning.

7.5 Security Implementation

To ensure the system is secure, the following security measures are implemented:

- **Authentication & Authorization** using **JWT**.
- **Password Hashing** with **bcrypt.js** for secure user credentials.
- **Role-Based Access Control (RBAC)** to restrict permissions.
- **Secure API Endpoints** using **HTTPS** and **CORS policies**.

- **Database Security** by limiting external access and using **environment variables** for sensitive data.

7.6 Performance Optimization

To enhance the system's performance:

- **Caching techniques** using **Redis** or in-memory caching.
- **Load balancing** for efficient request handling.
- **Code Optimization** by reducing redundant operations.
- **Database Indexing** for faster queries.
- **CDN (Content Delivery Network)** for serving static assets efficiently.

7.7 System Maintenance and Monitoring

Once the system is live, continuous monitoring and maintenance ensure stability and reliability. This includes:

- **Error logging** using tools like **Winston or LogRocket**.
- **System Health Monitoring** with **Prometheus & Grafana**.
- **Regular Software Updates** to patch vulnerabilities.
- **User Feedback Collection** for continuous improvements.

8. Software Maintenance

Software maintenance is the process of modifying, updating, and improving the system after deployment to ensure its stability, security, and efficiency. This phase is crucial for addressing issues, adding new features, and adapting to changing user needs.

8.1 Types of Software Maintenance

Software maintenance can be categorized into four main types:

1. **Corrective Maintenance**
    - Fixes bugs, errors, and security vulnerabilities.
    - Addresses system crashes, unexpected failures, and incorrect functionalities.
2. **Adaptive Maintenance**
    - Updates the system to meet changing hardware, software, and business requirements.
    - Adapts to new operating systems, browser versions, or external APIs.
3. **Perfective Maintenance**
    - Enhances performance and usability based on user feedback.
    - Optimizes code, improves database queries, and speeds up system response.
4. **Preventive Maintenance**
    - Proactively identifies and resolves potential issues before they cause failures.
    - Includes regular system health checks, security updates, and database optimizations.

8.2 Maintenance Process

The software maintenance process follows a structured approach:

1. **Issue Identification & Logging**
    - Users report bugs or request improvements.
    - Issues are logged in a tracking system (e.g., **Jira, Trello, or GitHub Issues**).
2. **Analysis & Prioritization**
    - Issues are classified as **critical, high, medium, or low priority**.

- The development team evaluates the impact and feasibility of solutions.

3. **Implementation & Testing**
    - Developers fix bugs or introduce enhancements.
    - Changes undergo **unit testing, integration testing, and regression testing**.
4. **Deployment & Monitoring**
    - Updates are deployed in **staging** before production release.
    - The system is monitored for stability, and user feedback is collected.

8.3 Tools & Technologies for Maintenance

- **Version Control:** Git & GitHub for tracking changes.
- **Error Logging & Monitoring:** LogRocket, Sentry, or Prometheus.
- **Automated Testing:** Jest, Mocha, or Cypress.
- **CI/CD Pipelines:** GitHub Actions or Jenkins for automated deployment.

8.4 Security Patching & Updates

Regular security updates are necessary to prevent vulnerabilities. Key security measures include:

- **Updating dependencies** to the latest stable versions.
- **Patching security flaws** in third-party libraries.
- **Monitoring API and database security** using firewalls and encryption.

8.5 Performance Optimization & Scaling

To ensure system efficiency as user traffic grows:

- **Database indexing** for faster queries.
- **Load balancing** to distribute traffic evenly.
- **Caching mechanisms** (Redis) for reducing load times.
- **Server auto-scaling** for handling increased demand.

8.6 User Support & Feedback Integration

- **Help Desk & Ticketing System** for resolving user queries.
- **Feedback Collection** through surveys and analytics.
- **Regular Updates** based on evolving user needs and technology trends.

9. Conclusion

The development of **Fit Hub** has successfully tackled the key health management challenges outlined in the problem statement. By conducting thorough system analysis, validating operational and technical feasibility, and implementing a modular and scalable architecture, the platform now stands as a **comprehensive, secure, and user-centric health solution**:

Key Highlights of Fit Hub:

- **MERN Stack Implementation**
  Developed using the **MongoDB, Express.js, React.js, and Node.js** stack to ensure performance, flexibility, and scalability across all components
- **Role-Based Access Control**
  Integrated support for **Admin**, **Physician**, and **User** roles ensures clear functionality, security boundaries, and streamlined workflows
- **Secure Authentication**
  Uses **Express-session-based authentication** and **bcrypt** hashing to protect sensitive user credentials and session data.
- **Robust Database Design**
  MongoDB schema structures store BMI records, diet plans, appointments, and progress logs efficiently with optimized querying and indexing
- **Extensive System Testing**
  Includes **validation testing**, **black-box testing**, and **white-box testing**, ensuring all modules—from BMI calculators to doctor consultations—function reliably under different scenarios
- **Collaboration and Version Control**
  Utilizes **Git and GitHub** for source control, allowing effective collaboration, clean version history, and streamlined updates.

10. Future Enhancements

To ensure continuous growth, feature expansion, and user satisfaction, **Fit Hub** is designed with a forward-thinking roadmap. The following enhancements are planned for future versions to further improve **performance, usability, and personalization**:

1. **AI-Powered Recommendations**
    - Integrate **machine learning algorithms** to:
        1. Suggest **personalized diet plans** based on user progress, preferences, and health trends
        2. Identify potential **health risks** using historical BMI and lifestyle data
        3. Recommend workouts and schedule adjustments tailored to user behavior
2. **Mobile App Development**
    - Launch a **cross-platform mobile app** using **React Native** or **Flutter** to enable users to:
        1. Track progress on the go
        2. Receive push notifications for meals, appointments, and updates
        3. Access diet plans, BMI calculators, and book doctor appointments from mobile devices
3. **Performance Optimization**
    - **Server-side caching** (e.g., Redis) for faster BMI and diet plan retrieval.
    - **Lazy loading and code-splitting** in the frontend to improve speed.
    - **Optimized database indexing** for faster access to appointments and progress logs
4. **Enhanced Security Features**
    - **Two-Factor Authentication (2FA) to enhance login security**.
    - **Role-Based Access Control (RBAC) with advanced permissions for doctors, users, and admins**.
    - Regular **vulnerability scans and penetration testing** to maintain compliance and trust.
5. **Integration with Third-Party Services**
    - **Google Fit**, **Apple Health**, or **fitness wearables** for real-time health tracking.
    - **Email and SMS APIs** for appointment reminders and health alerts.

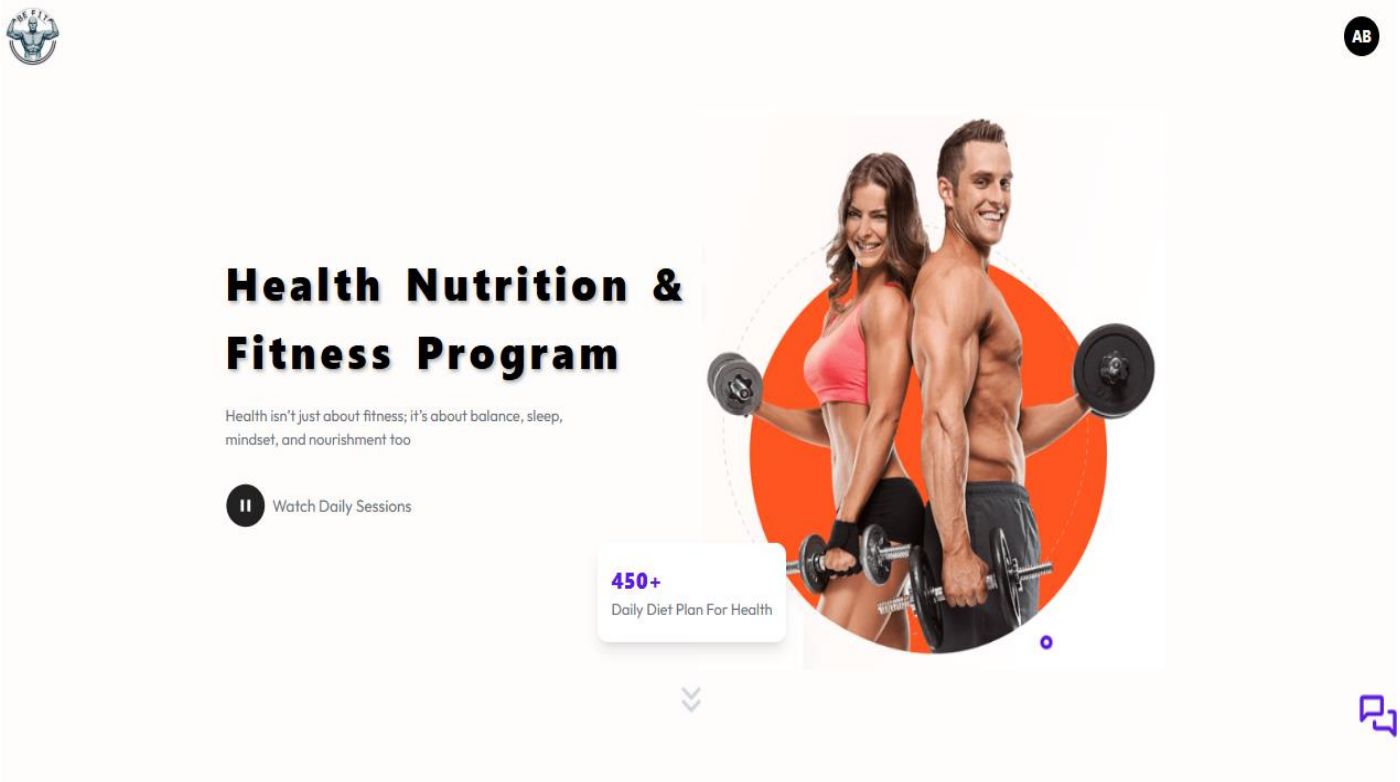
- **Social logins** (Google, Facebook) for simplified user registration

6. **Scalability & Cloud Deployment**
    - **Enable auto-scaling and load balancing**.
    - Ensure **high availability**, **failover support**, and **automated backups** for user data.
7. **Automated Testing & DevOps**
    - Establish a robust **CI/CD pipeline** for efficient code deployment and updates.
    - **Unit testing** for core features (BMI, diet generation).
    - **Integration and end-to-end testing** for workflows
    - **Version rollback capabilities** for error recovery
8. **User Feedback & Analytics**
    - User progress reports.
    - Diet adherence metrics
    - Health improvement trends
    - **Real-time feedback collection** via chatbots and surveys.
    - **In-app ratings** for doctor consultations and diet plans

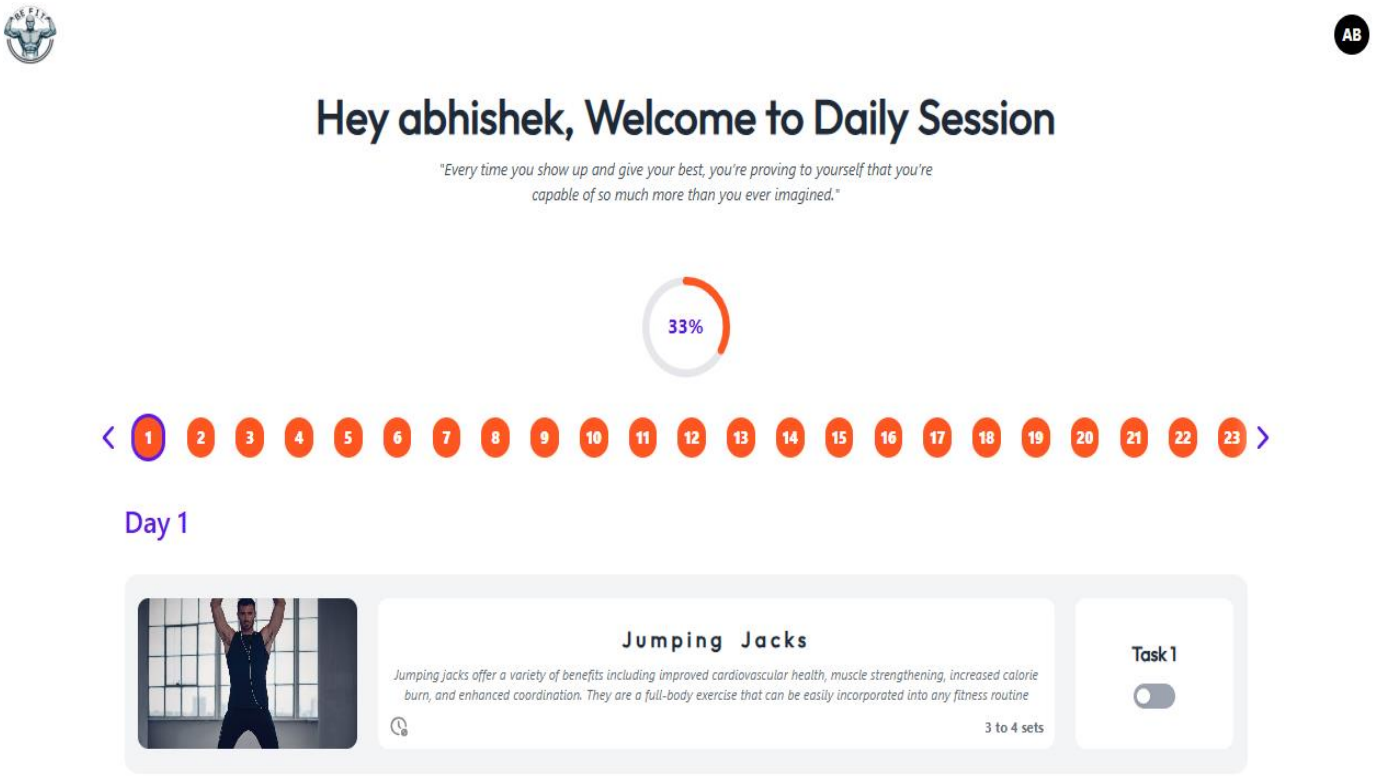
11.APPENDIX

11.1 SCREENSHOT

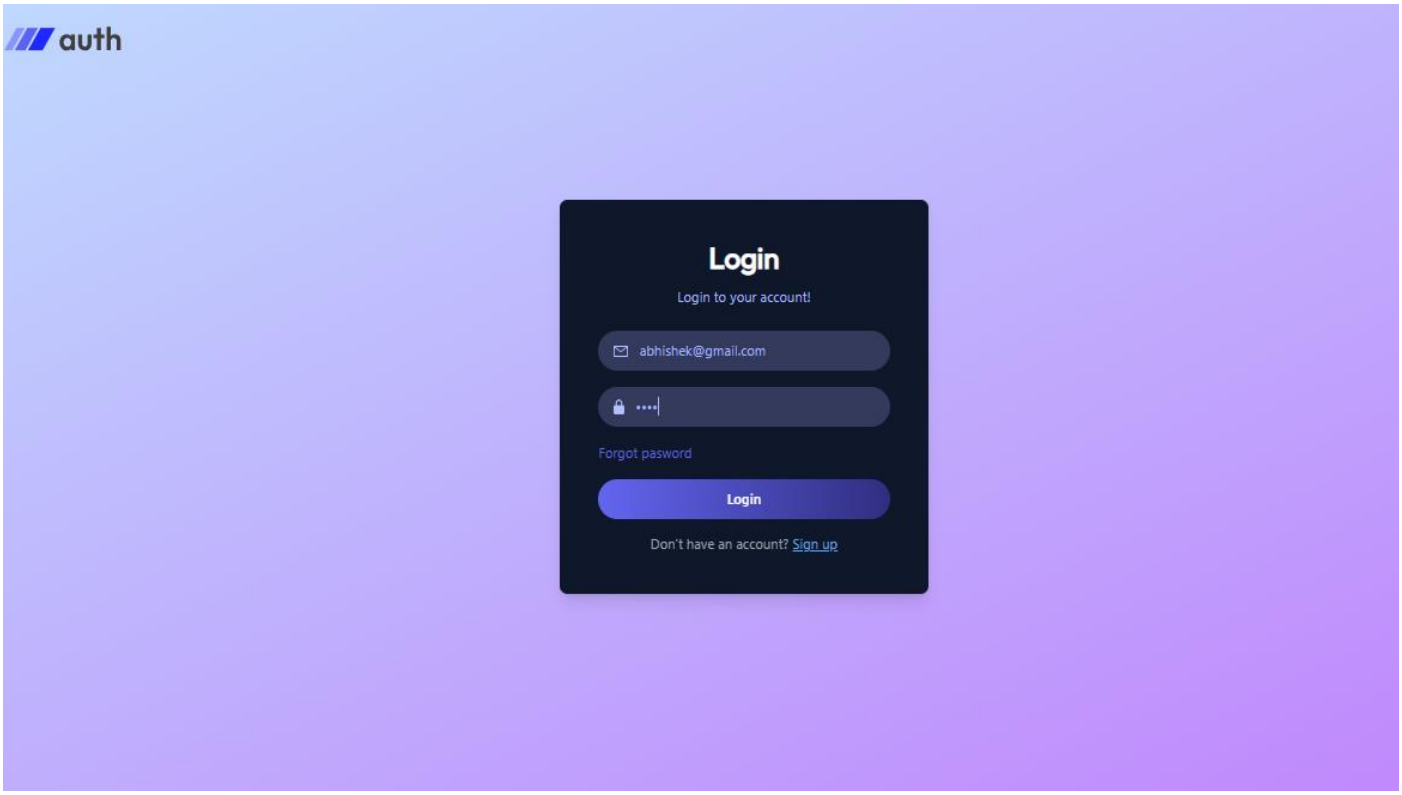
1.Home



2.Workout List



3.Login Page



5. Workout Mannage

Day 1



Jumping Jacks

Jumping jacks offer a variety of benefits including improved cardiovascular health, muscle strengthening, increased calorie burn, and enhanced coordination. They are a full-body exercise that can be easily incorporated into any fitness routine

3 to 4 sets

Task 1



pushups

Push-ups are a versatile exercise with numerous physical benefits. They improve upper body strength, core stability, and cardiovascular health, while also contributing to bone health and potentially aiding in weight management. Additionally,..

3 sets

Task 2



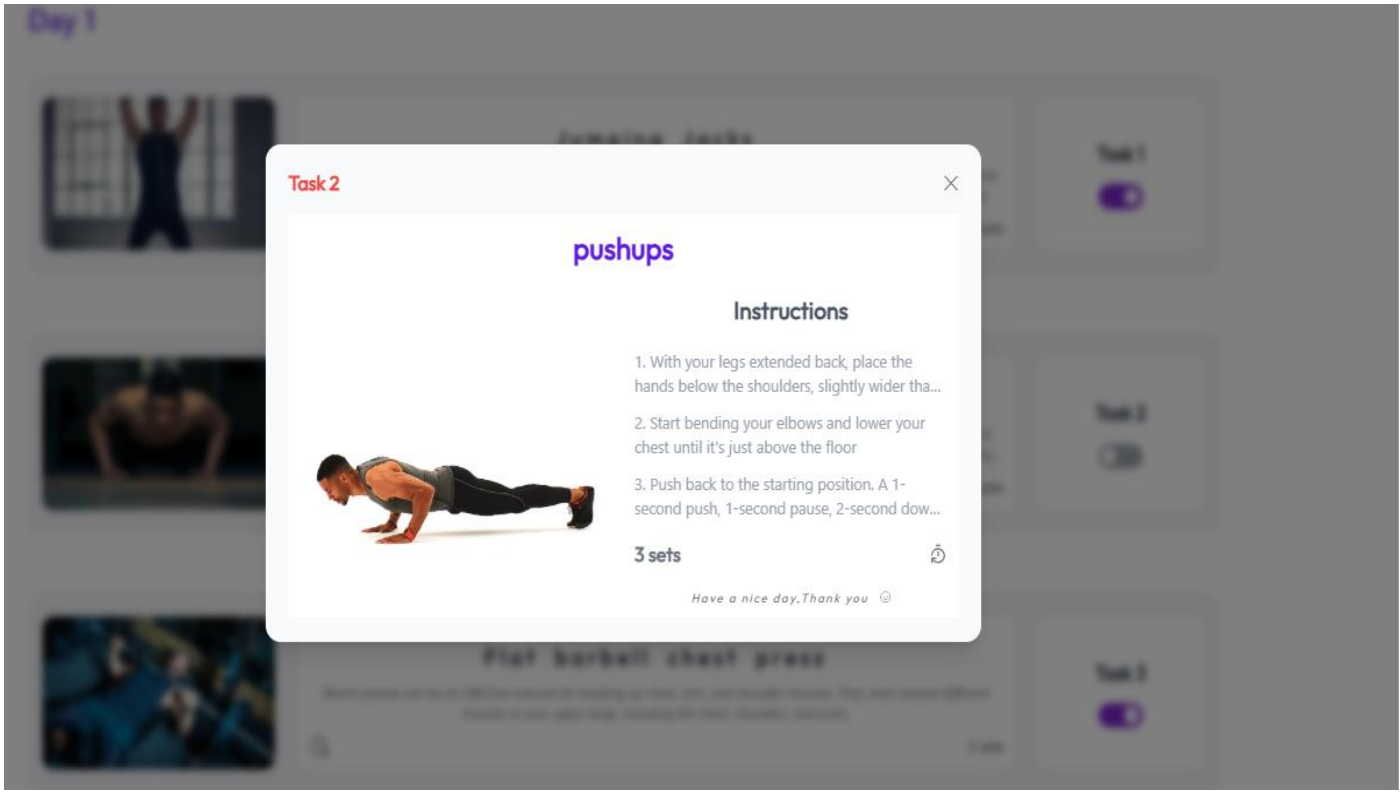
Flat barbell chest press

Bench presses can be an effective exercise for building up chest, arm, and shoulder muscles. They work several different muscles in your upper body, including the chest, shoulders, and arms.

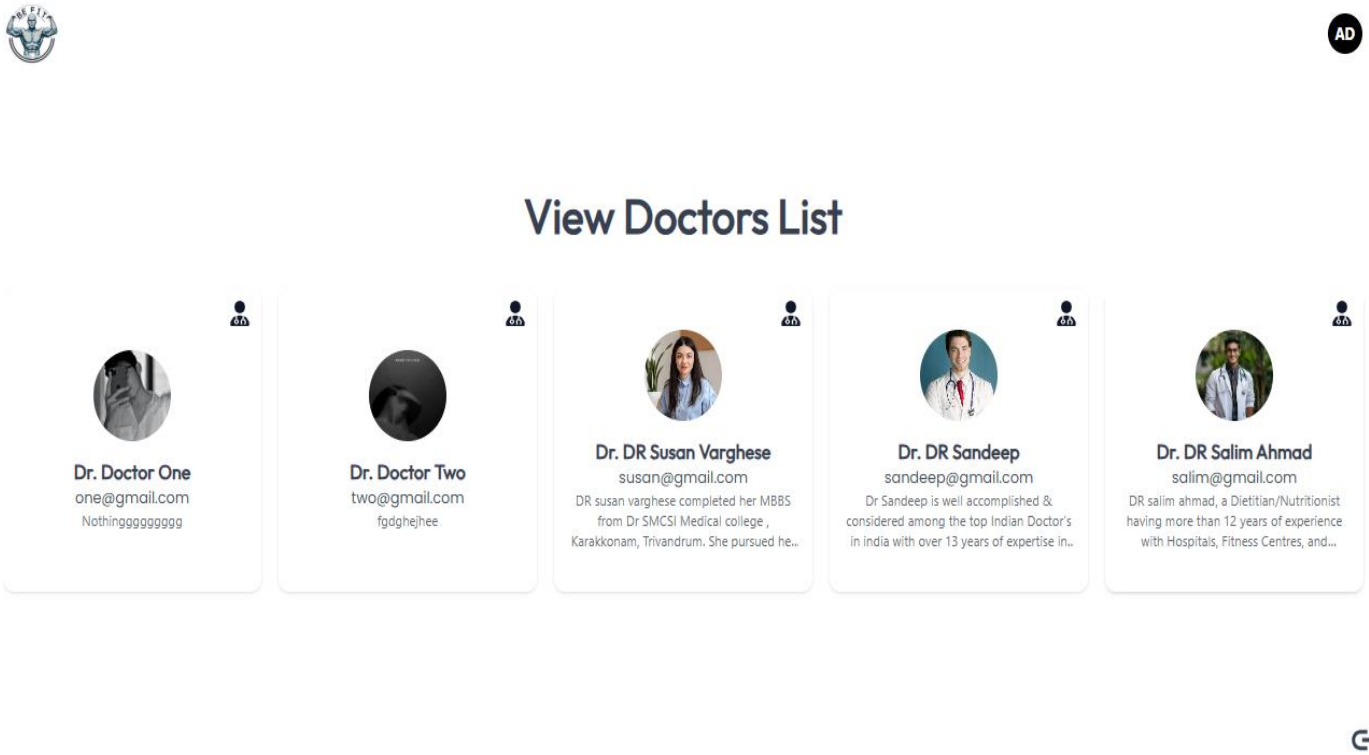
3 sets

Task 3

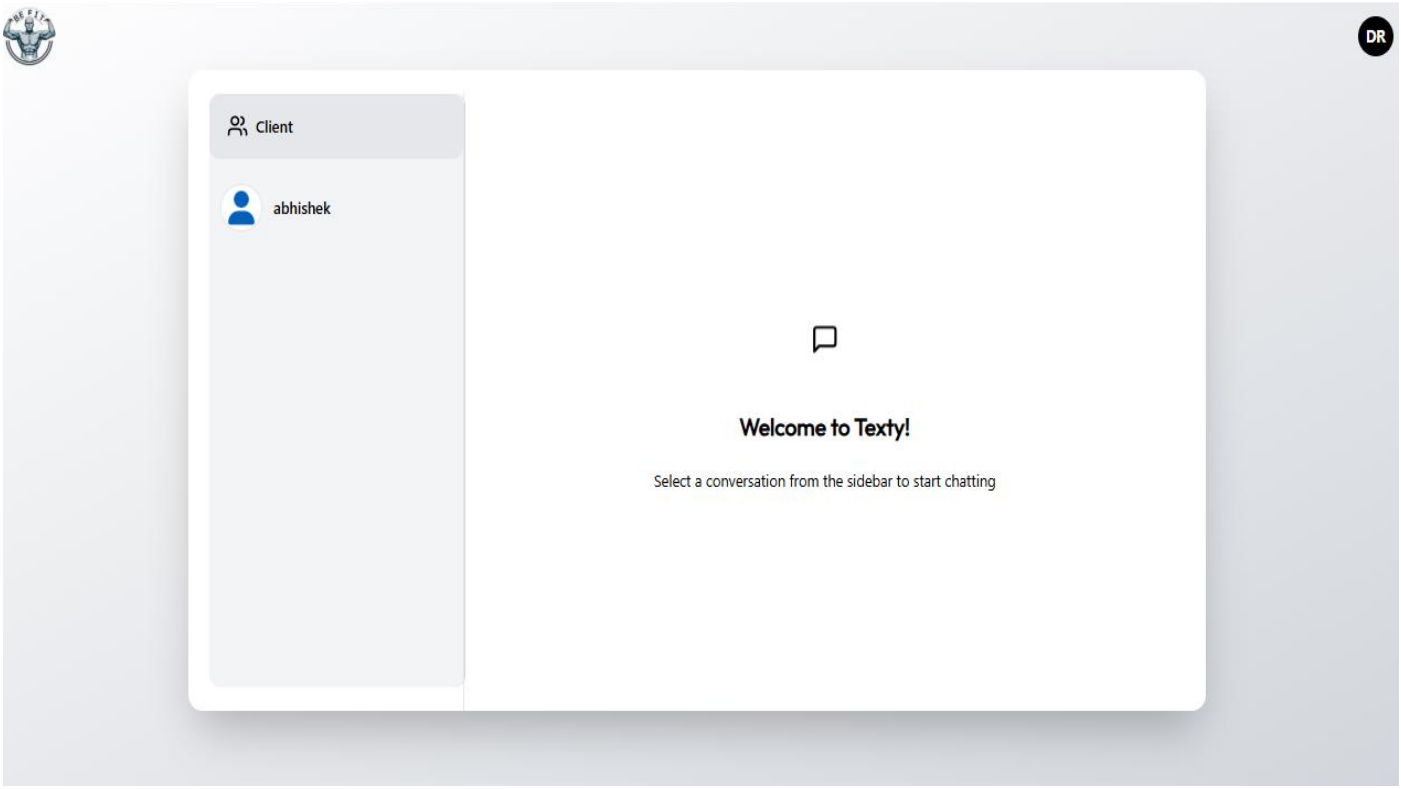
6. Post Workout



7. Doctors List



7.Chat Page


DR
Client
abhishek
Welcome to Texty!
Select a conversation from the sidebar to start chatting

11.2 SAMPLE CODE

- Datbase Schema

```
import mongoose, { Schema } from 'mongoose';

const WorkoutSchema = new Schema(
  {
    author: {
      type: String,
      default: 'admin',
    },
    wkdays: {
      type: [
        {
          day: {
            type: String,
            required: true,
          },
          workouts: {
            type: [
              {
                name: { type: String, default: "", required: true },
                inst: [{ type: String }],
                rep: { type: String, required: true },
                description: { type: String, required: true },
                workoutImage: { type: String, default: null }, // ✅ image filename
                workoutGif: { type: String, default: null },   // ✅ gif filename
              },
            ],
            default: [],
          },
        },
      ],
      default: [],
    },
  },
```

```
    { timestamps: true }
);

const WorkoutList = mongoose.model('Workout_List', WorkoutSchema);

export default WorkoutList;
```

- User Route

```
import express from 'express';
import { LoginValidation, registerValidation } from '../middlewares/authValidation.js';
import { getUserDetails, isUserauthenticatedornot, login, logout, register, resetPassword,
    sendRestOtp, sendVerificationOtp, verifyEmail,updateBMIratio,

updateWorkPage,getUserWorkoutList,setUserTasks,assignDoctor,getDoctorMessages,sendDoctorMe
ssages } from '../middlewares/authControllers.js';
import { isuserauth } from '../middlewares/isUserAuth.js';
import UserModel from '../models/userModels.js';

const router = express.Router();

router.post('/register', registerValidation, register);
router.post('/login', LoginValidation, login);
router.get('/logout', logout);
// router.post('/send-otp', isuserauth, sendVerificationOtp);
// router.post('/verify-email', isuserauth, verifyEmail);
// router.post('/reset-password', resetPassword);



router.get('/is-auth', isuserauth, isUserauthenticatedornot);
router.post('/send-reset-otp', sendRestOtp);
router.get('/user-details', isuserauth, getUserDetails)
router.get('/getProfile',isuserauth,async(req,res)=>{
    const userDetails = await UserModel.findOne({_id:req.body.userId});
    res.json(userDetails)
})
router.patch('/updateBMIratio',isuserauth,updateBMIratio)
```

```
router.patch("/updateWorkPage/:page",isuserauth,updateWorkPage)
router.get("/getUserWorkoutList/:page",isuserauth,getUserWorkoutList)
router.get("/setUserTasks/:wrkId/:day",isuserauth,setUserTasks)

router.get("/assignDoctor",isuserauth,assignDoctor)
router.get("/getDoctorMessages",isuserauth,getDoctorMessages)
router.patch("/sendDoctorMessages",isuserauth,sendDoctorMessages)

export default router;
```

- Root Route

```
import express from 'express';
import cors from 'cors';
import 'dotenv/config';
import cookieParser from 'cookie-parser';
import { connectDB } from './config/mongodb.js';
import path from 'path';
import { fileURLToPath } from 'url';

import UserRouter from './routes/userRoute.js';
import AdminRouter from './routes/AdminRoute.js';
import DoctorRouter from './routes/doctor.route.js'
import { runAdminSetup } from './middlewares/AdminSetup.js';

const app = express();
const port = process.env.PORT || 4000;

// Resolve __dirname for ES module
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Allow frontend (React) to access backend with cookies
const allowedOrigins = ['http://localhost:5173'];
app.use(cors({ origin: allowedOrigins, credentials: true }));

app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ limit: '50mb', extended: true }));
```

```
app.use(cookieParser());

// 🔥 Serve uploaded images
app.use('/uploads', express.static(path.join(process.cwd(), 'uploads')));

// Connect to DB
connectDB().then(runAdminSetup)

// Routes
app.use('/', UserRouter);
app.use('/admin', AdminRouter);
app.use('/doctor', DoctorRouter)

app.listen(port, () => console.log(` ✅ Server running on port ${port}`));
```

- First Page (React JS)

```
import React, { useContext, useState, useRef, useEffect } from 'react';
import { AppContent } from '../../contexts/AppContext';
import { useNavigate } from 'react-router-dom';
import { assets } from '../../assets/assets';
import { TbClockCancel } from "react-icons/tb";
import { FaX } from 'react-icons/fa6';
import { FaTrash } from "react-icons/fa";
import { BiSolidBookAdd } from "react-icons/bi";
import { GoSmiley } from "react-icons/go";
import { LuTimerReset } from "react-icons/lu";
import { IoIosArrowBack, IoIosArrowForward } from "react-icons/io";
import axios from 'axios';
import { toast } from 'react-hot-toast';
import { PiMaskSadLight } from "react-icons/pi";

const DailySessions = () => {
    const [toggles, setToggles] = useState([]); // Initially empty
    //const [toggles, setToggles] = useState(Array(10).fill(false));
    const [showMessagePopup, setShowMessagePopup] = useState(false);
    const [selectedIndex, setSelectedIndex] = useState(null);
    const [showFormPopup, setShowFormPopup] = useState(false);
```

```
const [formData, setFormData] = useState({
    workoutName: "",
    instruction1: "",
    instruction2: "",
    instruction3: "",
    repetitions: "",
    description: "",
    workoutImage: null,
    workoutGif: null,
});
const [daySelectedIndex, setDaySelectedIndex] = useState(1)
const [Drefresh, setDrefresh] = useState(false)
const [workoutList, setWorkoutList] = useState(null)
const [completepersentage, setCompletionPersantage] = useState(0)


const { userData, backendUrl, refresh, setRefresh } = useContext(AppContent);

const navigate = useNavigate();


const handleToggle = async (e, index, wrkId) => {
    e.stopPropagation();
    const updatedToggles = [...toggles];
    updatedToggles[index] = !updatedToggles[index];
    setToggles(updatedToggles);

    try {
        const { data } = await axios.get(
            `${backendUrl}/setUserTasks/${wrkId}/${daySelectedIndex}`, { withCredentials: true }
        );
        console.log("setUserTasks", data);
        if (data?.message === "Pending") {
            toast(`Task ${(index + 1)} marked as pending 💤`, {
                icon: "⏳",
            });
```

```
        } else {
            toast.success("Now Task " + (index + 1) + " " + data?.message)
        }

        const truethValues = updatedToggles.filter(item => item === true);

        const completionPercentage = (truethValues.length / workoutList.workouts.length) * 100;

        console.log("Boyyyeeee", completionPercentage);


        setCompletionPersantage(completionPercentage)

    } catch (error) {
        console.log(error);

    }
};

const handleCardClick = (index) => {
    setSelectedIndex(index);
    setShowMessagePopup(true);
};


const radius = 50;
const stroke = 8;
const normalizedRadius = radius - stroke * 0.5;
const circumference = normalizedRadius * 2 * Math.PI;

// Prevent NaN by setting a fallback
const safeCompletion = isNaN(completepersentage) ? 0 : completepersentage;
const strokeDashoffset = circumference - (safeCompletion / 100) * circumference;

const scrollRef = useRef(null);

const scrollLeft = () => {
```

```
    scrollRef.current.scrollBy({ left: -200, behavior: 'smooth' });
};

const scrollRight = () => {
    scrollRef.current.scrollBy({ left: 200, behavior: 'smooth' });
};

useEffect(() => {
    //console.log("userData userData", userData);
    getUserWorkoutList(daySelectedIndex)
    //console.log("Selected index changed to:", daySelectedIndex);
}, [daySelectedIndex, Drefresh]);



const handleInputChange = (e) => {
    const { name, value, files } = e.target;

    if (files) {
        setFormData({ ...formData, [name]: files[0] });
    } else {
        setFormData({ ...formData, [name]: value });
    }
};

const handleSubmit = async (e) => {
    e.preventDefault();

    try {
        const formDataToSend = new FormData();
        formDataToSend.append("workoutName", formData.workoutName);
        formDataToSend.append("instruction1", formData.instruction1);
        formDataToSend.append("instruction2", formData.instruction2);
        formDataToSend.append("instruction3", formData.instruction3);
        formDataToSend.append("repetitions", formData.repetitions);
        formDataToSend.append("description", formData.description);
        if (formData.workoutImage) {
```

```
        formDataToSend.append("workoutImage", formData.workoutImage);
      }
      if (formData.workoutGif) {
        formDataToSend.append("workoutGif", formData.workoutGif);
      }

      const { data } = await axios.post(
        `${backendUrl}/admin/addWorkoutList/${daySelectedIndex}`,
        formDataToSend,
        {
          withCredentials: true,
          headers: {
            "Content-Type": "multipart/form-data",
          },
        }
      );

      setFormData({
        workoutName: "",
        instruction1: "",
        instruction2: "",
        instruction3: "",
        repetitions: "",
        description: "",
        workoutImage: null,
        workoutGif: null,
      });

      toast.success(data?.message);
      setDrefresh(!Drefresh);
      setShowFormPopup(false);

    } catch (error) {
      console.log(error);
    }
  };
```

```
const userchoosenDau = async (index) => {
    setDaySelectedIndex(index)

    try {
        const { data } = await axios.patch(
            `${backendUrl}/updateWorkPage/${index}`, {}, { withCredentials: true }
        );
        //  console.log(data);

    } catch (error) {
        console.log(error);


    }

}

const getUserWorkoutList = async (page) => {
    try {
        const { data } = await axios.get(
            `${backendUrl}/getUserWorkoutList/${page}`,
            { withCredentials: true }
        );

        if (!data?.success) {
            console.log("Workout not found or error:", data);
            setWorkoutList(null); // optional: clear previous workouts
            setToggles([]);
            setCompletionPersantage(0);
            return;
        }

        setWorkoutList(data.workouts);

        const workouts = data?.workouts?.workouts || [];
        const activedToggles = workouts.map(item => !!item.isActived);
        const completedCount = activedToggles.filter(Boolean).length;
```

```
        const completionPercentage = (completedCount / workouts.length) * 100;

        setToggles(activedToggles);
        setCompletionPersantage(completionPercentage);

        console.log("Toggles:", activedToggles);
        console.log("Completion %:", completionPercentage.toFixed(2));

    } catch (error) {
        console.error("Error fetching workouts:", error);
        // Optionally show a toast
        toast.error("Failed to load workout list");
    }
};



const deleteWorkout = async (day, id) => {
    //console.log("Day", day, "ID", id);
    try {
        const { data } = await axios.patch(
            `${backendUrl}/admin/deletWorkout`, { day: day, Id: id }, { withCredentials: true }
        );
        // console.log("Deleteee, data");
        setDrefresh(!Drefresh)
        toast.success(data?.message)

    } catch (error) {
        console.log(error);

    }

}

useEffect(() => {
    setDaySelectedIndex(userData?.wrkpage)
    getUserWorkoutList(userData?.wrkpage)
```

```
    //e.log("Selected index changed to:", daySelectedIndex);
  }, []);

  return (
    <div className='flex flex-col items-center justify-center px-4 text-gray-800 relative p-6'>
      {/* Greeting Section */}
      <div className='relative w-auto flex flex-col items-center justify-center h-auto mt-10 lg:mt-20'>
        <div className='text-center flex flex-col items-center justify-center'>
          <h2 className='text-3xl sm:text-5xl font-semibold mb-4'>Hey {userData?.role === "admin" ? "Admin" : userData.name}, Welcome to {userData?.role === "admin" ? "Mannaging" : "Daily"}  Session</h2>
          <p className='text-gray-600 mb-8 max-w-xl italic text-sm sm:text-base'>
            {userData?.role === "admin" ? '"Here you can add or delete user workout details, upload the workout name, repetitions, description, workout image, and workout GIF. A white background leads to a better user experience."' :
              '"Every time you show up and give your best, you’re proving to yourself that you’re capable of so much more than you ever imagined."'}
          </p>
        </div>
        {
          userData?.role === "user" && (
            <div className='mt-6'>
              <div className="w-[130px] h-[130px] relative flex items-center justify-center text-center">
                <svg height={radius * 2} width={radius * 2} className="transform -rotate-90">
                  {/* Background circle */}
                  <circle
                    stroke="#e5e7eb"
                    fill="transparent"
                    strokeWidth={stroke}
                    r={normalizedRadius}
                    cx={radius}
                    cy={radius}
                  />
                  {/* Foreground progress circle */}
```

```
                        <circle
                            stroke="#ff5520"
                            fill="transparent"
                            strokeWidth={stroke}
                            strokeLinecap="round"
                            strokeDasharray={`${circumference} ${circumference}`}
                            strokeDashoffset={strokeDashoffset}
                            r={normalizedRadius}
                            cx={radius}
                            cy={radius}
                        />
                    </svg>
                    {/* Percentage text */}
                    <div className="absolute inset-0 flex items-center justify-center text-xl font-bold text-[#5d1be5]">
                        {`${Math.round(safeCompletion)}%`}
                    </div>
                </div>
            </div>
        )
    }


</div>

<div className='h-[50px] px-5 flex items-center justify-center gap-3 my-5 w-full lg:w-[1380px] relative '>
    <div
        className='h-full w-12 sm:w-16  bg-gradient-to-r from-[#ffff] absolute left-0
        flex items-center justify-center z-10 cursor-pointer hover:scale-125'
        onClick={scrollLeft}
    >
        <IoIosArrowBack size={35} className='text-[#5d1be5]' />
    </div>

    <div
        ref={scrollRef}
```

```
            className='flex items-center gap-6 overflow-x-auto max-w-7xl hide-scrollbar px-2 ml-5 sm:ml-0'
          >
            {Array(30).fill().map((_, index) => (
              <div
                key={index}
                className={`w-[10px] lg:w-[30px] px-3 lg:px-4 py-1 lg:py-2 rounded-full bg-[#ff5520] flex items-center justify-center
        text-[16px] ${daySelectedIndex === index + 1 ? "border-4 border-[#5d1be5]" : ""} text-white  cursor-pointer font-extrabold whitespace-nowrap`}
                onClick={() => {
                  userchoosenDau(index + 1)
                }}
              >
                {index + 1}
              </div>
            ))}
          </div>

          <div
            className='h-full w-12 sm:w-16 p-2 bg-gradient-to-l from-[#ffff] absolute right-0 flex items-center
             justify-center z-10 cursor-pointer hover:scale-125'
            onClick={scrollRight}
          >
            <IoIosArrowForward size={35} className='text-[#5d1be5]' />
          </div>
        </div>

        {/* Workout Sessions List */}
        <div className='w-full h-auto flex flex-col items-center justify-center p-5  gap-9' >

          <div className=' max-w-5xl lg:max-w-7xl w-full  flex items-center'>
            <h1 className='text-lg sm:text-3xl font-semibold text-[#5d1be5]'>Day {daySelectedIndex}</h1>
          </div>
          {
```

```
            workoutList && workoutList?.workouts && workoutList?.workouts.length != 0 ? <>
              {workoutList?.workouts.map((item, index) => (
                <div
                  key={index}
                  onClick={() => handleCardClick(index)}
                  className='relative w-full cursor-pointer max-w-5xl lg:max-w-7xl h-auto bg-gray-100 rounded-2xl flex flex-col md:flex-row items-center justify-between gap-6 px-4 py-6 hover:bg-gray-200 mb-5 opacity-0 animate-show'
                  style={{ animationDelay: `${index * 0.2}s` }}
                >
                  {/* Image */}
                  <div className="flex-shrink-0">
                    <img className='w-[250px] h-[150px] rounded-xl object-cover' src={`${backendUrl}/uploads/workouts/${item?.workoutImage}`} alt="Pushup" />
                  </div>

                  {/* Middle Content */}
                  <div className='flex-1 w-[250px] h-[150px] bg-white rounded-xl flex flex-col items-center justify-center relative p-4'>
                    <h2 className='text-md sm:text-[22px] font-extrabold mb-2 text-center' style={{ letterSpacing: "5px", wordSpacing: "8px" }}>
                      {item?.name}
                    </h2>
                    <p className='text-gray-500 text-[12px] sm:text-sm max-w-[800px] italic text-center line-clamp-3 lg:line-clamp-2'>
                      {item?.description}
                    </p>
                    <div className='pt-8 lg:pt-5 w-full'>
                      <TbClockCancel size={22} color='gray' className='absolute bottom-3 left-3' />
                      <div className='absolute bottom-3 right-3'>
                        <p className='text-gray-500 font-bold text-xs sm:text-sm'>{item?.rep}</p>
                      </div>
                    </div>
                  </div>
```

```
                    {/* Right Side Toggle */}
                    <div className='w-[250px] md:w-[180px] h-[150px] rounded-xl bg-white flex flex-col items-center justify-center p-2'>
                        <h2 className='text-lg sm:text-xl font-semibold mb-4'>Task {index + 1}</h2>
                        <div className="flex items-center justify-center">
                            {
                                userData?.role != "admin" ? (
                                    <button
                                        onClick={(e) => handleToggle(e, index, item?._id)}
                                        className={`w-12 h-6 flex items-center rounded-full p-1 transition-colors duration-300 ease-in-out ${toggles[index] ? 'bg-purple-700' : 'bg-gray-400'
                                            }`}
                                    >
                                        <div className={`bg-white w-4 h-4 rounded-full shadow-md transform duration-300 ease-in-out ${toggles[index] ? 'translate-x-6' : ''
                                            }`} />
                                    </button>
                                ) : (
                                    <button disabled
                                        onClick={(e) => handleToggle(e, index)}
                                        className={`opacity-20 w-12 h-6 flex items-center rounded-full p-1 transition-colors duration-300 ease-in-out ${toggles[index] ? 'bg-purple-700' : 'bg-gray-400'
                                            }`}
                                    >
                                        <div className={`bg-white w-4 h-4 rounded-full shadow-md transform duration-300 ease-in-out ${toggles[index] ? 'translate-x-6' : ''
                                            }`} />
                                    </button>
                                )
                            }

                        </div>
                    </div>

                    {/* Trash Icon (for user role) */}
                    {userData?.role === "admin" && (
```

```
                        <div
                            className='absolute top-4 right-4 font-extrabold cursor-pointer text-red-
500 z-50'
                            onClick={(e) => {
                                e.stopPropagation(); // ⛔ stop click from bubbling to parent
                                deleteWorkout(daySelectedIndex, item._id);
                            }}
                        >
                            <FaTrash className='hover:scale-110' size={16} />
                        </div>
                    )}
                </div>
            ))}

        </> :
            <>
                <div className='flex items-center justify-between gap-7 '>
                    <h2 className='text-xl sm:text-3xl font-semibold' style={{ letterSpacing: "5px"
}}>Nothing Found</h2>
                    <PiMaskSadLight size={40} />
                </div>
            </>
        }
        {
            userData?.role === "admin" &&
            <div onClick={() => setShowFormPopup(true)} className='absolute bottom-1
lg:bottom-5 right-8 cursor-pointer  lg:right-10  text-[40px] lg:text-[46px] pt-6 lg:pt-2 text-
[#5d1be5]'>
                <BiSolidBookAdd className='hover:scale-110 ' />
            </div>
        }

    </div>

    {/* Message Popup Modal */}
    {showMessagePopup && (
```

```
        <div className="fixed top-0 left-0 bottom-0 right-0 flex items-center justify-center z-50
">
            <div className="absolute inset-0 bg-black bg-opacity-50 backdrop-blur-sm"
onClick={() => setShowMessagePopup(false)}></div>

            <div className="relative bg-gray-50 rounded-2xl shadow-xl p-6 w-[90%] max-w-3xl
z-50 animate-fadeIn overflow-y-auto ">
              <div className="flex justify-between items-center mb-4">
                <h2 className="text-xl font-bold text-red-500">Task {selectedIndex + 1}</h2>
                <button className="text-gray-500 hover:text-red-500 text-xl" onClick={() =>
setShowMessagePopup(false)}>✕</button>
              </div>
              <div className="mx-auto p-4 text-center bg-white">
                <h2 className="text-3xl font-bold text-
[#5d1be5]">{workoutList?.workouts[selectedIndex].name}</h2>
              </div>
              <div className='w-full flex flex-col lg:flex-row items-center justify-center bg-
white'>
                <div className='w-[70%] lg:w-[50%] p-3'>
                  <img
src={`${backendUrl}/uploads/workouts/${workoutList?.workouts[selectedIndex].workoutGif}`}
className='w-full h-auto' alt="Push-up" />
                </div>
                <div className='w-[70%] lg:w-[50%] p-3 flex flex-col items-center justify-center
gap-5'>
                  <div>
                    <h2 className="text-2xl font-bold text-gray-600">Instructions</h2>
                  </div>
                  <div>
                    <ol className='list-decimal list-inside text-gray-400 space-y-3'>
                      {
                        workoutList?.workouts[selectedIndex].inst.map((inItem, index) => (
                          <li className='line-clamp-2' key={index}>{index + 1}. {inItem}</li>
                        ))
                      }
                    </ol>
                  </div>
```

```
                    <div className='w-full flex items-center justify-between'>
                      <h2 className='text-xl font-bold text-gray-500 line-clamp-1'>
                        {workoutList?.workouts[selectedIndex].rep}
                      </h2>
                      <LuTimerReset size={20} color='gray' />
                    </div>
                    <div>
                      <h2 className='text-xs  italic text-gray-500 flex items-center justify-center gap-3' style={{ letterSpacing: "2px" }}>
                        Have a nice day,Thank you <GoSmiley />
                      </h2>
                    </div>
                  </div>
                </div>
              </div>

          )}

          {showFormPopup && (
            <div className="fixed top-0 left-0 bottom-0 right-0 flex items-center justify-center z-50">
              <div className="absolute inset-0 bg-black bg-opacity-50 backdrop-blur-sm" onClick={() => setShowFormPopup(false)}></div>
              <form
                onSubmit={handleSubmit}
                className="relative bg-white rounded-2xl shadow-xl p-6 w-[90%] max-w-2xl z-50 animate-fadeIn space-y-4"
                encType="multipart/form-data"
              >
                <div className="flex justify-between items-center mb-2">
                  <h2 className="text-xl font-bold text-gray-700">Customize Workout</h2>
                  <div className="flex items-center justify-center gap-20">
                    <h2 className="text-lg font-bold text-gray-700 flex">TAB NO. {daySelectedIndex}</h2>
                    <button type="button" className="text-gray-500 hover:text-red-500 text-xl" onClick={() => setShowFormPopup(false)}>✕</button>
```

```
            </div>
          </div>

          <div className="flex items-center justify-center gap-4">
            <div className="flex flex-col items-center justify-center gap-4">
              <label className="lg:font-extrabold">Workout Name</label>
              <input name="workoutName" value={formData.workoutName}
onChange={handleInputChange} placeholder="Workout Name" className="w-full border p-2
rounded-lg" required />
            </div>
            <div className="flex flex-col items-center justify-center gap-4">
              <label className="lg:font-extrabold">Workout Image</label>
              <input name="workoutImage" type="file" onChange={handleInputChange}
className="w-full border p-2 rounded-lg" />
            </div>
          </div>

          <input name="instruction1" value={formData.instruction1}
onChange={handleInputChange} placeholder="Instruction 1" className="w-full border p-2
rounded-lg" required />
          <input name="instruction2" value={formData.instruction2}
onChange={handleInputChange} placeholder="Instruction 2" className="w-full border p-2
rounded-lg" required />
          <input name="instruction3" value={formData.instruction3}
onChange={handleInputChange} placeholder="Instruction 3" className="w-full border p-2
rounded-lg" required />
          <textarea name="description" value={formData.description}
onChange={handleInputChange} placeholder="Workout Description" className="w-full lg:h-
[100px] border p-2 rounded-lg" required />

          <div className="flex items-center justify-center gap-4">
            <div className="flex flex-col items-center justify-center gap-4">
              <label className="lg:font-extrabold">Repetitions</label>
              <input name="repetitions" value={formData.repetitions}
onChange={handleInputChange} placeholder="Repetitions (e.g., 15 / 3)" className="w-full border
p-2 rounded-lg" required />
            </div>
```

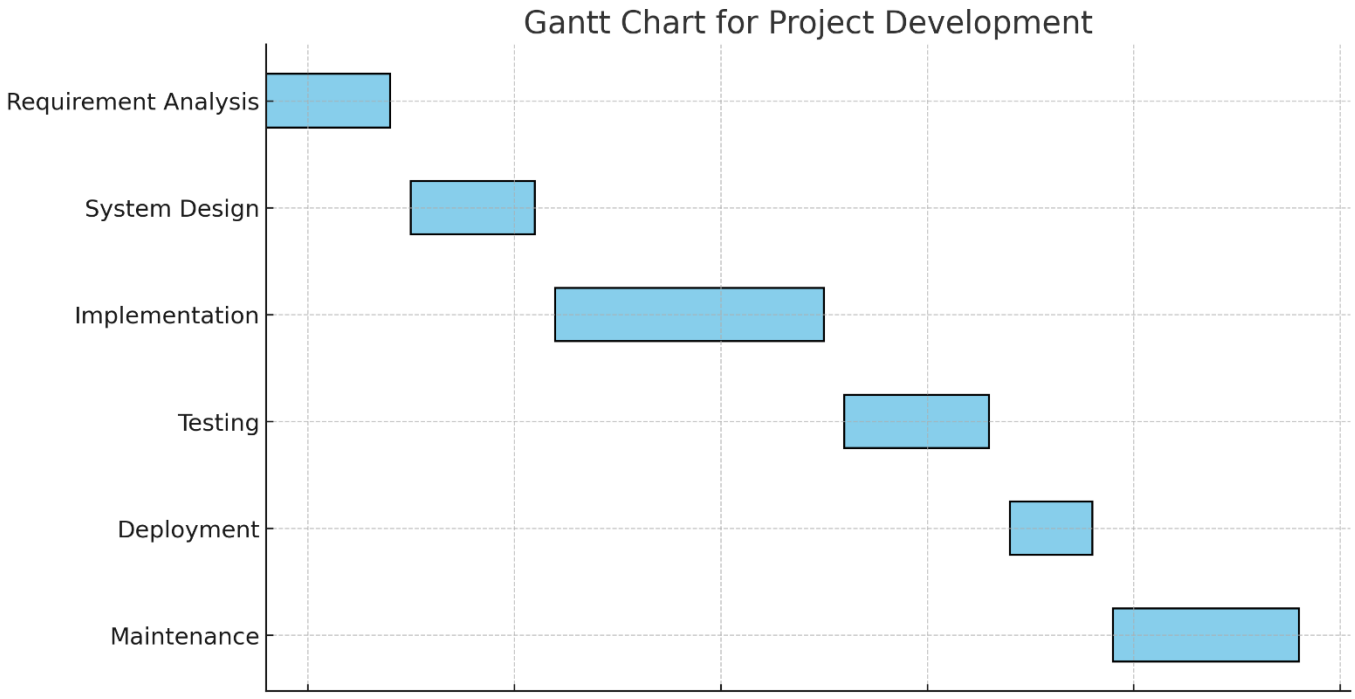
```
                    <div className="flex flex-col items-center justify-center gap-4">
                        <label className="lg:font-extrabold">Workout GIF</label>
                        <input name="workoutGif" type="file" onChange={handleInputChange}
                    </div>
                </div>

                <button type="submit" className="w-full bg-purple-700 text-white py-2 rounded-lg
                </form>
            </div>
        )}
    </div>
  );
};

export default DailySessions;
```

11.3 GANTT CHART

A **Gantt Chart** is a project management tool that outlines the schedule of tasks, their dependencies, and timelines. Below is a structured **Gantt Chart** for your **Shop Management System** project.



12. References

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
4. MongoDB Documentation. (n.d.). Retrieved from https://www.mongodb.com/docs/
5. Node.js Documentation. (n.d.). Retrieved from https://nodejs.org/en/docs/
6. React.js Documentation. (n.d.). Retrieved from https://react.dev/
7. Express.js Guide. (n.d.). Retrieved from https://expressjs.com/en/guide/routing.html
8. ISO/IEC 25010:2011. *Systems and Software Quality Requirements and Evaluation (SQuaRE)*. International Organization for Standardization.
9. Agile Manifesto. (2001). Retrieved from https://agilemanifesto.org/
10. Lucidchart. (n.d.). UML Diagramming Guide. Retrieved from https://www.lucidchart.com/pages/uml-diagram